Federated Learning-based Intrusion Detection System for SDN-enabled IoT Networks



PROJECT REPORT

Submitted by

Aaron B Ajay (1BM23CS003)
Aaryan Prakash (1BM23CS006)
Amal Roy (1BM23CS025)
Amogh Krishna JS (1BM23CS029)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Mr. Syed Akram

Assistant Professor,Department of CSE, BMSCE



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU - 560019

Academic Year - 2026

Certificate

This is to certify that the project report entitled **"Federated Learning-based Intrusion Detection System for SDN-enabled IoT Networks"** is a bonafide work carried out by **Aaron B Ajay (USN: 1BM23CS003), Aaryan Prakash (USN: 1BM23CS006), Amal Roy (USN: 1BM23CS025) and Amogh Krishna JS (USN: 1BM23CS029)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Computer Science and Engineering**, under my supervision during the academic year 2026.

Guide Signature	HoD signature	Principal Signature
(Mr. Syed Akram)	Dr. Kavitha Sooda	Dr. Bheemsha Arya

Name and signature of Examiners along with date

Examiner 1

Examiner 2

Abstract

Internet of Things (IoT) and Industrial IoT (IIoT) networks connect large numbers of lightly protected devices that are frequent targets of denial-of-service floods, scanning, web injection attacks, malware and man-in-the-middle attacks. A conventional intrusion detection system (IDS) collects all traffic centrally, cannot name attacks it was never trained on, and does not stop the attack. This project presents **FL-IDS**, an intrusion detection system for IoT/IIoT traffic that combines five components: a LightGBM gradient-boosting classifier that names known attack types; a client-side autoencoder that checks the traffic the classifier passes as normal and flags unusual traffic as anomalous, as a backstop for zero-day attacks; federated learning with the Flower framework, so that the autoencoder is trained across ten non-IID clients without sharing raw traffic; a server-side cosine-similarity trust filter that excludes poisoned client updates before trimmed-mean aggregation; and software-defined networking (SDN) mitigation that turns a verdict into an OpenFlow rule.

The system is evaluated on the Edge-IIoTset dataset (2.2 million packets, 15 classes). We first corrected two problems in the released data: a placeholder-formatting artefact that separated benign from attack traffic with an AUROC of 1.0 (independently found here, and first reported by Galal, 2026), and column-shifted MITM rows, which we found and repaired from the authors' own capture. The classifier was then improved with a service-port feature, payload and URI content features, and flow features computed with CICFlowMeter's 120-second active timeout, together with square-root class weighting. Trained on 1.43 million packets and tested on 397 thousand packets from connections it never saw (a flow-grouped split of all 1.99 million unique packets), accuracy rose from 96.4% to **99.7%** and macro F1 from 0.889 to **0.983–0.985** across three seeds, meeting the project target of 95% for both.

The same features were integrated into the federated system, which was then re-evaluated on all 1.99 million unique packets, split by connection, with 10 and 20 client processes and three seeds. Known attacks are caught at 0.994–0.998 attack macro-recall. When each attack class was held out of training in turn, the cascade detected the unseen class at 0.895–0.944 on average, against 0.723–0.757 for the classifier alone. With the trust filter, known-attack recall stayed at 0.985–0.997 with up to 50% poisoned clients, while plain federated averaging's autoencoder collapsed. All 14 replayed

attack captures ended in a drop rule confirmed from the switch's flow table, 12 of them classified as the right attack type. A full training run takes 18 to 31 minutes and the models decide on a packet in a median 0.64 ms (model time only). Limitations, including capture-level rather than inline mitigation, a weaker trust filter and autoencoder at full scale, and single-seed stealth-attacker, pruning and clipping tests, are stated openly.

DECLARATION

We, hereby declare that the Major Project Phase-2 work entitled "Federated Learning-based Intrusion Detection System for SDN-enabled IoT Networks" is a bonafide work and has been carried out by us under the guidance of Mr. Syed Akram, Assistant Professor, Department of Computer Science and Engineering, B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements of the degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi. I further declare that, to the best of my knowledge and belief, this project has not been submitted either in part or in full to any other university for the award of any degree.

Candidate details:

SL. NO.	Student Name	USN	Student's Signature
1	Aaron B Ajay	1BM23CS003	
2	Aaryan Prakash	1BM23CS006	
3	Amal Roy	1BM23CS025	
4	Amogh Krishna JS	1BM23CS029	

Place: Bengaluru
Date:

Certified that these candidates are students of Computer Science and Engineering Department of B.M.S. College of Engineering. They have carried out the project work of titled "Federated Learning-based Intrusion Detection System for SDN-enabled IoT Networks" as Major Project Phase-2 work. It is in partial fulfillment for completing the requirement for the award of B.E. degree by VTU. The works is original and duly certify the same.

Guide Name: Mr. Syed Akram
Date:
Signature

Acknowledgment

We thank our guide, Mr. Syed Akram, Assistant Professor, Department of Computer Science and Engineering, for his guidance, feedback and encouragement throughout this project.

We thank Dr. Kavitha Sooda, Head of the Department of Computer Science and Engineering, and Dr. Bheemsha Arya, Principal, B.M.S. College of Engineering, for providing the facilities and support needed to carry out this work.

We thank the faculty members and staff of the Department of Computer Science and Engineering for their help, and the authors of the Edge-IIoTset dataset for making their data and captures publicly available.

Finally, we thank our families and friends for their constant support.

Contents

1 Introduction **1**
1.1 Background of the Study 1
1.2 Research Motivation 2
1.3 Objective 2
1.4 Scope 3
1.5 SDG Justification 3
1.6 Existing System 3
1.7 Proposed System 4
1.8 Work Plan 4

2 Literature Survey **7**
2.1 Overview 7
2.2 Detailed Survey 7
2.3 Research Gaps 9
2.4 Mapping of Project Objectives with Research Gaps 10

3 Requirement Analysis and Specification **11**
3.1 Stakeholders 12
3.2 Functional Requirements 12
3.3 Non-Functional Requirements 13
3.4 System Requirements 15
3.4.1 Hardware requirements 15
3.4.2 Software requirements 16
3.5 Data Requirements 16
3.6 Constraints and Limitations 17
3.7 Assumptions 18
3.8 Risks and Challenges 18

4 Designs **20**
4.1 System Architecture Overview 20
4.2 Module/Component Design 21
4.3 Detailed Design 22

4.3.1 One federated round 22
4.3.2 Cascade decision 23
4.3.3 Trust filter and aggregation 23
4.3.4 Phase B: from capture to rule 24
4.3.5 Classes and interfaces that matter 24
4.4 Data Design 24
4.4.1 Dataset and splits 24
4.4.2 Files written by a run 25
4.4.3 Record layout 25
4.4.4 Bundle validation 26

5 Methodology **27**
5.1 Tools and Techniques 27
5.2 Data Set with Explanation 28
5.2.1 Data-quality problems found and fixed 28
5.2.2 Preprocessing 29
5.2.3 New features for the classifier 29
5.3 Flowchart of Methodology from Start to End 31
5.4 Explanation of Methodology in Detail with Equations and Formulae . . 32
5.4.1 How much data each part uses 32
5.4.2 Non-IID partition and per-client normalisation 33
5.4.3 Stage 1: LightGBM classifier 34
5.4.4 Stage 2: autoencoder and threshold 34
5.4.5 Cascade decision rule 35
5.4.6 Trust filter and robust aggregation 35
5.4.7 Incremental classifier update 36
5.4.8 SDN mitigation 36
5.4.9 Evaluation metrics 37

6 Results and Discussion **38**
6.1 Presentation of Results 38
6.1.1 Classifier accuracy 38
6.1.2 The whole system on the full dataset, split by connection 42
6.1.3 Training time and time to decide on a packet 48
6.1.4 Boosting vs. a deep-learning classifier 49
6.1.5 Earlier results on the 60,000-row pool 50
6.2 Limitations of the Study 63
6.3 Conclusion 65

7 Conclusion **66**

REFERENCES **68**

PLAGIARISM **70**

I Appendix A: Snapshots 71

II Appendix B: Details of list of publications related to this project 73

III Appendix C: Details of Patent 74

IV Appendix D: Details of Funding 75

V Appendix E:POs and PSOs Mapped 76

List of Figures

4.1 Deployment view of FL-IDS: server, clients and the emulated network . 20
4.2 High-level design of FL-IDS 21
4.3 Order of messages in one federated round 23
4.4 How the dataset is divided; every connection goes to exactly one box . . 25

5.1 Flow chart of the methodology 32
5.2 Two-stage cascade decision rule 35
5.3 One federated round 36

6.1 Accuracy and macro F1 on held-out flows as each feature group is added 40
6.2 Per-class F1, base features vs. final classifier (flow-grouped split) 41
6.3 Row-normalised confusion matrix of the final classifier (flow-grouped split) 42
6.4 Earlier (60k pool, split by row) and full-pool (split by connection) results for the six runs 44
6.5 Zero-day detection by the cascade at full scale (one dot per seed) and the earlier 60k-pool mean 47
6.6 Ablation: known-attack and zero-day macro-recall, original vs. current features 51
6.7 Detection and malicious-client survival vs. share of malicious clients (130 features) 52
6.8 Zero-day detection per held-out class: boosting alone vs. cascade (130 features) 53
6.9 Cascade zero-day detection per held-out class: original 91 features vs. 130 features 53
6.10 Real run with 30% sign-flip attackers: trust score (left) and cosine similarity to the round's median update (right) 58
6.11 Benign validation loss with and without the per-client z-score clip . . . 60
6.12 Single-feature AUROC before the placeholder fix (red) and best single features after it (blue) 61
6.13 Autoencoder reconstruction error on held-out client test rows (10-client run); grey band: range of the clients' thresholds 62

6.14 Rows per client and class for the 10-client run (Dirichlet $\alpha = 0.3$, seed 42) 62
6.15 Phase B timeline for one replay of the Backdoor capture 63

1 Project results website (`https://aronnnn1.github.io/FL-IDS/`). A second, presentation-style site with animated diagrams is at `https://aronnnn1.github.io/FL-IDS-showcase/` . . 71

List of Tables

1.1 Project work plan 6

2.1 Mapping of objectives to research gaps 10

3.1 Stakeholders and their needs 12
3.2 Hardware used for development and evaluation 15
3.3 Software used 16
3.4 Risks, challenges and how each was handled 19

4.1 Packages and modules of FL-IDS 22
4.2 Files that make up a model bundle and a run record 25

5.1 Class distribution after cleaning (91 base features) 28
5.2 Class distribution of the final data (130 features) 31
5.3 Training and evaluation rows for each part (130 features, seed 42) . . . 33

6.1 Ambiguity ceiling per feature set (all rows, after de-duplication) 39
6.2 Accuracy and macro F1 as feature groups are added (LightGBM, all rows, 80/20) 39
6.3 Final classifier across seeds 40
6.4 Real multi-process Phase A on the full pool (1.89M rows), split by connection, 20 rounds, no attackers 43
6.5 The same runs at the earlier scale and now (macro-recall includes only attack types present in the test slices) 44
6.6 Recall per attack type, 10 clients, seed 42, full pool, by stage (held-out test slices) 45
6.7 Poisoning sweep, full pool 45
6.8 Ablation at 30% sign-flip attackers, full pool, seeds 42 / 43 / 44 46
6.9 Zero-day (leave one attack class out), full pool: mean over the held-out classes 46
6.10 Zero-day (leave one attack class out), full pool: mean over the held-out classes: per held-out class 46
6.11 Stealth attacker vs. sign-flip, full pool 47
6.12 Pruning during federated training, full pool (no attackers) 47

6.13 Per-client z-score clip, final-round benign validation loss 47
6.14 Phase B on every attack capture (10 clients, seed 42, full-pool bundle): class chosen, confidence, deciding stage, packets flagged, action 48
6.15 Time taken by each full-dataset run (`journey.json` of each bundle) . 49
6.16 Time for the models to decide on a packet (model time only, one thread) 49
6.17 Ablation at 30% malicious clients, 130 features [91 features] (MR = macro-recall) 50
6.18 Real multi-process Phase A runs (source: `results/real_runs/summary.md`) 54
6.19 Phase A stage timing (source: `results/journey/phase_a_journey.json`) 55
6.20 Phase B stage timing (source: `results/journey/phase_b_journey.json`) 55
6.21 Ablation at 30% sign-flip attackers, seeds 42 / 43 / 44 (source: results/seeds/summary.json) 56
6.22 Poisoning sweep, seeds 42 / 43 / 44 56
6.23 Zero-day detection of the unseen class, mean over 14 classes, seeds 42 / 43 / 44 57
6.24 Where the real-run macro-recall comes from (source: results/real_runs/¡run¿/, made by fl_ids.eval.bundle_report) 57
6.25 Real multi-process Phase A with sign-flip attackers (10 clients, 20 rounds, seed 42; source: runs/attack_runs/) 57
6.26 Adaptive (stealth) attacker vs. sign-flip, seed 42, 20 rounds (source: results/adaptive/adaptive_attack.csv) 58
6.27 Post-training magnitude pruning of the 10-client autoencoder, thresholds recalibrated (source: results/pruning/c10_s42/pruning.csv) 59
6.28 Pruning during federated training (server re-prunes each round, linear ramp over the first 10 rounds), seed 42, no attackers (source: results/pruning/pruning_during_fl.csv) 59
6.29 Benign validation loss with and without the per-client z-score clip (source: results/clip/clip_comparison.csv) 59
6.30 Per-class time-ordered split vs. connection-grouped split (source: experiments/accuracy/results/full_temporal_bal_s*.json) 60
6.31 Single-feature AUROC, attack vs. normal (source: results/leak/single_feature_auroc.json; after: 300,000-row sample) 61
6.32 Objectives and the evidence for each (whole dataset, three seeds unless stated) 65

1 Programme Outcomes Mapping with Project 76
2 Programme Specific Outcomes Mapping with Project 77

Chapter 1 Introduction

1.1 Background of the Study

Industrial and consumer Internet of Things (IoT/IIoT) deployments connect large numbers of low-power sensors, actuators, edge gateways and controllers. These devices speak lightweight protocols such as MQTT and Modbus/TCP, run with little or no host-based security, and are often installed in places that are physically hard to reach and slow to patch. They are therefore a common target for denial-of-service floods, scanning, injection attacks against web interfaces, malware such as backdoors and ransomware, and man-in-the-middle attacks.

A network intrusion detection system (IDS) watches traffic and labels it as benign or as a particular attack type. A conventional IDS sends all traffic to one central place for training and analysis. In IoT/IIoT settings this is a problem: raw traffic can reveal how a plant or a home operates, traffic volume is high, and different sites see very different mixes of traffic.

This project, **FL-IDS**, builds an intrusion detection system for IoT/IIoT traffic that combines five parts, each covering a gap left by the others:

1. a **LightGBM gradient-boosting classifier** that names known attack types;
2. a **client-side autoencoder** that checks the traffic the classifier calls normal and flags anything unlike benign traffic as *anomalous*, as a backstop for attacks that were never seen during training (zero-day attacks);
3. **federated learning** with the Flower framework, so the autoencoder is trained across many sites without raw traffic leaving any site;
4. a **cosine-similarity trust filter** on the server that screens the clients' model updates and excludes poisoned ones before aggregation;
5. **software-defined networking (SDN)** mitigation, where a verdict becomes a real OpenFlow rule (drop or rate-limit) on an Open vSwitch switch running in Mininet.

The system is evaluated on the public Edge-IIoTset dataset [1], which contains about 2.2 million labelled packets across normal traffic and 14 attack types.

1.2 Research Motivation

Three practical problems motivated the work.

Privacy and bandwidth. Packet captures from a factory floor or a smart building contain operational details that owners do not want to share. Federated learning lets each site train on its own data and share only model weights.

Attacks the classifier has never seen. A supervised classifier can only output the labels it was trained on. A new attack is either forced into a wrong known class or called normal. An anomaly detector trained only on benign traffic does not need attack labels, so it can catch some of these cases.

Poisoned training. In federated learning the server cannot see client data, so a compromised client can send a manipulated update. Without a defence, a few such clients can ruin the shared model. During this project, plain federated averaging with 30% sign-flipping clients drove the autoencoder's validation loss to about 9.2×10^{14} and its stand-alone detection to zero.

Finally, detection alone does not stop an attack. Linking the detector to an SDN controller lets a verdict change how the network forwards traffic.

1.3 Objective

1. **Malicious Client Detection & Filtering.** To develop a **malicious client detection mechanism** using anomaly-based or trust-based evaluation to identify and filter compromised participants in the federated network.

2. **Efficient Privacy-Aware FL Design.** To design a **lightweight and efficient federated learning framework** that balances privacy and performance while minimizing computational and communication overhead in resource-constrained IoT environments.

3. **Model Optimization for Scalability.** To enhance system scalability by applying **model optimization techniques such as weight pruning**, reducing computational and communication overhead.

4. **Semi-Supervised & Anomaly-Based Detection.** To integrate **semi-supervised learning and Autoencoder-based anomaly detection** to effectively identify abnormal or potentially poisoned patterns without relying entirely on labeled data.

1.4 Scope

The project covers offline federated training and evaluation ("Phase A") and a live mitigation demonstration on an emulated network ("Phase B"). Training uses 10 or 20 simulated clients, run either in one process for the evaluation sweeps or as separate operating-system processes (one per client) talking to a real Flower server. The main results use the whole Edge-IIoTset dataset (1.99 million unique packets), split by connection. Mitigation is demonstrated by replaying real attack captures from the dataset through Mininet with `tcpreplay`.

The following are outside the scope: deployment on physical IoT hardware, inline per-packet blocking of a live stream (classification happens once tshark has extracted features from a capture), adaptive attackers other than the sign-flip attacker, and a systematic literature review to establish novelty.

1.5 SDG Justification

SDG 9: Industry, Innovation and Infrastructure. FL-IDS enhances the security of network infrastructure and supports smart systems: IIoT networks run manufacturing, power and transport, and the system detects attacks on them, resists poisoned training and blocks attackers through SDN flow rules.

SDG 16: Peace, Justice and Strong Institutions. The project strengthens cybersecurity and protects sensitive digital data: raw traffic never leaves a site during training, and attacks are detected and blocked.

1.6 Existing System

Most IDS work on Edge-IIoTset and similar datasets trains a single centralised model, often a deep neural network or a tree ensemble, on the authors' preprocessed CSV file, and reports accuracy on a random split of packets. These systems have the following limitations:

- all raw traffic must be collected in one place;
- they can only output known attack labels, so a new attack is missed or mislabelled;
- federated variants that do exist usually use plain federated averaging, which a small number of malicious clients can break;
- detection is not linked to any enforcement mechanism;

- the reported accuracy is often inflated, because random packet splits put packets of the same connection in both training and test sets, and because of formatting artefacts in the released CSV (Section 5.2.1).

1.7 Proposed System

FL-IDS uses a two-stage cascade. Every packet first goes to the LightGBM classifier. If it predicts an attack with probability at least 0.7, that label is final. Otherwise the packet goes to the client's autoencoder. If the reconstruction error is above that client's threshold (the 97th percentile of error on its own benign validation data), the packet is labelled *anomalous*; otherwise it is benign.

The autoencoder is trained with federated learning across 10 or 20 non-IID clients. On every round the server's trust filter compares each client's weight update with the coordinate-wise median update, excludes outliers, updates that point away from the consensus and clients with low running trust, clips the remaining update norms and combines them with a trimmed mean.

The trained models are saved as a self-describing bundle. In Phase B they classify replayed attack traffic on a Mininet network, and the verdict is sent to an os-ken OpenFlow 1.3 controller, which installs a drop rule or a rate-limiting meter on the switch.

This report also adds flow- and content-level features to the classifier (Chapter 5), which raise it from 96.4% accuracy and 0.887 macro F1 to the levels reported in Chapter 6.

1.8 Work Plan

Phase	**Work**	**Period**	**Status**
1	Literature survey on intrusion detection, federated learning and SDN; identification of the problem	January – February 2026	Done
2	Study of the research gaps and selection of the novel contributions	March 2026	Done
3	Comparison of many machine-learning and deep-learning classifiers on Edge-IIoTset; choice of boosting for the first stage	March – April 2026	Done

Phase	**Work**	**Period**	**Status**
4	Project scaffolding, configuration, test framework	April 2026	Done
5	Data pipeline: cleaning, encoding, Dirichlet partition, per-client normalisation; label-leak fix and MITM capture repair	April – May 2026	Done
6	LightGBM bootstrap classifier and incremental update	May 2026	Done
7	Core federated loop with the Flower framework	May – June 2026	Done
8	Robustness layer: trust filter, trimmed mean, sign-flip attacker	June 2026	Done
9	Training on the real Edge-IIoTset data	July 2026	Done
10	Evaluation: ablation, poisoning sweep, zero-day experiment	July – August 2026	Done
11	SDN mitigation with Mininet, Open vSwitch and os-ken	August 2026	Done
12	Streamlit dashboard	August 2026	Done
13	End-to-end Phase A $\rightarrow$ Phase B run	September 2026	Done
14	Classifier accuracy improvement (flow and content features)	September 2026	Done
15	Integration of the new features into the federated system and the SDN demo; full re-evaluation	late September – early October 2026	Done
16	Follow-up experiments: 20 clients, repeat seeds, adaptive attacker, pruning, time-ordered split	October 2026	Done
17	Whole system re-run on the full dataset, split by connection (10 and 20 clients, three seeds); class split, training time and per-packet decision time	October 2026	Done
18	Results showcase website, project report and presentation	late October – 3 November 2026	Done

Phase	**Work**	**Period**	**Status**
19	Paper writing and submission	November 2026 onwards	In progress

Table 1.1: Project work plan

Chapter 2 Literature Survey

This chapter reviews published work on the techniques FL-IDS combines: federated intrusion detection, robust aggregation against poisoned updates, two-stage classifier-plus-anomaly-detector designs, and coupling of learning-based detection with SDN enforcement. It then identifies the gaps this project addresses.

2.1 Overview

Federated learning (FL) was introduced by McMahan *et al.* [9] as a way to train a shared model across many clients by averaging locally trained weights (FedAvg), so raw data stays on the clients. Intrusion detection was one of the first security applications of FL, and the area has grown to the point where Hernandez-Ramos *et al.* [4] published a systematic review of FL-based IDS in *ACM Computing Surveys*. Their review covers the models, datasets, aggregation functions and libraries used in this literature. It shows that a federated autoencoder or a federated deep classifier is a well-established idea in itself, and that the open problems lie in non-IID data, poisoning, and real deployment.

FedAvg is fragile when some clients are malicious. Yin *et al.* [10] analysed coordinate-wise median and trimmed-mean aggregation and showed that they tolerate a bounded fraction of Byzantine clients, but break down once attackers reach the trimmed fraction, or 50% for the median. Later work moved to the *direction* of updates: comparing each update with a reference using cosine similarity.

2.2 Detailed Survey

Mothukuri *et al.*, 2022 [2]. This work proposes federated anomaly detection for IoT networks using gated recurrent unit (GRU) models trained on decentralised device data, with an ensembler that combines updates from several sources. It shows that federated training can match centralised accuracy while keeping data on devices. It uses no supervised first stage, no defence against poisoned updates, and no network-level enforcement.

Li *et al.*, DeepFed, 2021 [3]. DeepFed builds an intrusion detection model for industrial cyber-physical systems from a convolutional neural network and a GRU, trained

with FL across several industrial sites. Model parameters are protected with a Paillier-cryptosystem-based protocol. DeepFed protects updates in transit but does not filter malicious ones, and it has no zero-day backstop or SDN mitigation.

Ferrag *et al.*, Edge-IIoTset, 2022 [1]. This paper releases the dataset used in this project. It is collected on a seven-layer IoT/IIoT testbed with more than ten device types and covers 14 attacks in five groups (DoS/DDoS, information gathering, man-in-the-middle, injection and malware). The authors explicitly position it for both centralised and federated learning and report baseline results for both. Chapter 5 shows that the released CSV has formatting artefacts which inflate results trained on it directly.

Galal, 2026 [13]. This preprint shows that in Edge-IIoTset the "field does not apply" value is serialised as `0` or `0.0` depending on the source capture, so it encodes the label, and builds a leakage-free benchmark for precision-agriculture IDS using random stratified and leave-one-device-out splits. FL-IDS found and fixed the same artefact independently and later, and credits it to this work. Galal does not report the column-shifted MITM rows, and uses no flow features, connection-grouped split, federated training or SDN mitigation.

Cao *et al.*, FLTrust, 2021 [5]. FLTrust gives the server a small clean "root" dataset and computes its own reference update. Each client update gets a trust score from its cosine similarity to that reference, with negative similarity clipped to zero, and its magnitude is normalised to the reference. FL-IDS borrows the rule that an update pointing *away* from the reference (cosine < 0) is excluded. It uses the coordinate-wise median of the round's updates as the reference instead of a root dataset.

Xu *et al.*, SignGuard, 2022 [6]. SignGuard shows that median- and distance-based defences can be circumvented by carefully crafted poisoning, and that the element-wise sign of an update together with magnitude and similarity statistics is effective for filtering malicious gradients. This supports pairing cosine similarity with a norm check, which FL-IDS does with norm clipping, against the sign-flipping attacker used in its evaluation.

Zhang *et al.*, 2023 [7]. This paper proposes a two-stage intrusion detector: LightGBM with focal loss makes a first decision, and the reconstruction residual of an autoencoder makes a second decision on samples the first stage called normal. It reports above 90% accuracy, recall and F1 on NSL-KDD and UNSW-NB15. This is the direct precedent for the boosting-first cascade used in FL-IDS. FL-IDS differs in training the autoencoder federatedly and protecting it with a trust filter.

El Houda *et al.*, MiTFed, 2023 [8]. MiTFed lets several network domains collaboratively train an attack detector with FL, uses SDN to enforce mitigation, and uses a blockchain to make the collaboration verifiable. It is the closest precedent for the "FL decides, SDN enforces" structure of FL-IDS. MiTFed does not use a two-stage cascade with an anomaly backstop.

Ke *et al.*, LightGBM, 2017 [11]. LightGBM is a gradient-boosted decision-tree library that uses gradient-based one-side sampling and exclusive feature bundling to train quickly on large tabular data. Tree ensembles are a strong baseline for tabular network-flow features, which FL-IDS confirms: on the same 500,000-row sample, LightGBM reached 0.887 macro F1 against 0.622 for a comparably sized neural network.

Sharafaldin *et al.*, CIC-IDS2017, 2018 [14]. This work introduced the CIC-IDS2017 dataset, whose features are computed per flow by CICFlowMeter (packet counts, rates, durations, payload statistics). Flow features of this kind are standard in network IDS. FL-IDS adopts the same idea, including CICFlowMeter's 120-second active timeout for splitting long sessions into flow records.

2.3 Research Gaps

From the survey, the following gaps were identified.

1. **No single system combines all five parts.** Federated anomaly detection, cosine-based robust aggregation, a boosting-first cascade and FL with SDN enforcement each exist in some paper, as pairs or triples. A handful of targeted searches found no system combining all five. (This is not a systematic review, so novelty is stated cautiously.)

2. **Zero-day detection is rarely measured per class.** Most federated IDS papers report accuracy on known classes only. The value of an anomaly-detection backstop shows up only when each attack class is held out in turn.

3. **Robust aggregation under non-IID data.** Cosine-similarity defences assume honest updates are similar to each other. Under strong label skew, honest clients disagree too, and the literature reports that such defences degrade. This needs to be measured, not assumed.

4. **Inflated results on the released Edge-IIoTset CSV.** Work that trains directly on the authors' preprocessed CSV inherits its formatting artefacts. The random packet-level split also puts packets of the same connection in both training and test sets.

5. **Weak detection of application-layer attacks.** With only the released packet-header features, SQL injection, XSS, file upload, password attacks and HTTP floods are hard to tell apart from each other and from benign traffic, because most of their packets are content-free TCP acknowledgements.

6. **Detection rarely drives verified enforcement.** Many papers stop at a classification score. Few show a detection producing a flow rule that is then read back from the switch.

2.4 Mapping of Project Objectives with Research Gaps

Research gap	**Objective addressing it**
1. No system combines all five parts	All four objectives together: trust-filtered federated learning (1, 2), a compact model (3) and an autoencoder backstop (4), integrated with a boosting classifier and SDN mitigation
2. Zero-day detection rarely measured	Objective 4: autoencoder-based anomaly detection, measured by holding out each of the 14 attack classes
3. Robust aggregation under non-IID data	Objective 1: cosine/MAD/EMA-trust filtering of client updates, tested from 0% to 50% malicious clients
4. Inflated results on the released CSV	Supporting work for all objectives: placeholder-leak fix, MITM capture repair and a flow-grouped evaluation split
5. Weak application-layer detection	Supporting work: service-port, content and flow features for the classifier (Section 5.2.3)
6. Detection not linked to enforcement	Objective 2 (practical FL design): Phase B turns a verdict into an OpenFlow rule verified in the switch's flow table

Table 2.1: Mapping of objectives to research gaps

Chapter 3 Requirement Analysis and Specification

This chapter lists who the system is for, what it must do, the qualities it must have, the hardware, software and data it needs, and the constraints, assumptions and risks that shaped the work.

3.1 Stakeholders

Stakeholder	What they need from FL-IDS
Site operators (factory, building or campus owners running IoT/IIoT devices)	Detection of attacks on their own traffic without sending raw packet captures to anyone else; few false alarms; a clear verdict they can act on.
Security analysts	A label and a confidence for each alert, the stage that raised it (classifier or autoencoder), and a way to confirm alerts so that the classifier can learn from them.
Network administrators	Mitigation that is expressed as ordinary OpenFlow rules on a software switch, can be read back from the switch, and can be undone.
Federation coordinator (the server operator)	Protection against clients that send poisoned model updates, and a record of which clients were excluded in which round and why.
Project team and guide	Reproducible runs, logged numbers, and an honest statement of what the system does not do.
Examiners and reviewers	A report, presentation and results website in which every figure can be traced to a logged run.
Researchers who reuse the work	The data-quality findings, the connection-level split, the code, the configuration files and the model bundles.

Table 3.1: Stakeholders and their needs

3.2 Functional Requirements

FR1 **Data preparation.** Load the Edge-IIoTset CSV, repair the column-shifted MITM rows from the authors' own capture, canonicalise the categorical placeholder values, remove duplicate rows, one-hot encode the categorical fields, and derive the service-port, content and flow features (130 features in all).

FR2 **Non-IID partitioning.** Hold out a stratified 5% server calibration set and split the rest across $N = 10$ or $N = 20$ clients with a Dirichlet($\alpha = 0.3$) label-skew partition. Whole connections stay together: each client gets its own training, benign-validation and test slices, normalised with a scaler fitted only on that

client's training data.

FR3 **First-stage classification.** Train a LightGBM multi-class classifier on the server's calibration set, broadcast it to clients every round, and update it every 3 rounds with up to 50 analyst-confirmed alerts per surviving client.

FR4 **Second-stage anomaly detection.** On each client, train an autoencoder only on traffic the classifier passes as normal, and set a per-client threshold at the 97th percentile of benign validation reconstruction error, recomputed every round.

FR5 **Federated training.** Run a Flower server and the client processes for 20 rounds; clients send only weight updates and error histograms, never raw traffic.

FR6 **Poisoning defence.** On every round, exclude client updates that are cosine-similarity MAD outliers, that point away from the median update, or that come from clients with EMA trust below 0.5. Clip surviving update norms to $2\times$ the median and aggregate them with a 15% trimmed mean.

FR7 **Cascade decision.** Label each record as a known attack type, *anomalous* or benign using one shared decision rule for evaluation, dashboard and SDN.

FR8 **Model bundle.** Save the classifier, autoencoder weights, per-client scalers and thresholds, cascade settings, metrics, timing and provenance as a self-describing bundle, and reject inconsistent bundles on load.

FR9 **SDN mitigation.** Replay a capture through Mininet, extract features with tshark, classify, and send the verdict to an os-ken controller, which installs a drop rule (confidence ≥ 0.85) or an OpenFlow meter (confidence ≥ 0.5). Verify the rule from the switch's own flow table.

FR10 **Evaluation.** Produce the ablation table, poisoning-resistance sweep, zero-day leave-one-class-out results, classifier accuracy results, training times and per-packet decision times as CSV/JSON files and figures.

FR11 **Dashboard.** Show live training state (rounds, trust scores, surviving clients, reconstruction error, classifier version) and the live mitigation state in a Streamlit web dashboard.

3.3 Non-Functional Requirements

- **Privacy:** raw traffic must not leave a client during federated training. The only exception is the capped, analyst-confirmed alerts used for the classifier update, which is documented as a trade-off.

- **Accuracy:** the classifier must reach at least 95% accuracy and at least 0.95 macro F1 on held-out data, under an evaluation that keeps whole connections out of training.
- **Robustness:** known-attack recall must stay stable when up to 40% of clients are malicious.
- **Reproducibility:** every run is seeded (seed 42 by default, with seeds 43 and 44 as repeats); LightGBM runs deterministically; all tunable values live in configuration files (`configs/`).
- **Resource limits:** a full 10-client run on the whole dataset must complete on one virtual machine (Table 3.2); the per-client footprint was measured at about 0.4–0.8 GB of RAM.
- **Speed:** the models must decide on a packet fast enough that classification is not the bottleneck of a software switch; the measured figures are in Section 6.1.3.
- **Maintainability:** the code is split into modules by component, with 244 automated tests.
- **Honesty of reporting:** every number in this report comes from an actual logged run; superseded numbers are not reused.

3.4 System Requirements

3.4.1 Hardware requirements

Item	Specification used
Host machine	Windows 11 workstation running VMware Workstation (all later runs); an Apple MacBook Pro (M5) with UTM was used for early development and the first 60,000-row runs
Guest operating system	Ubuntu 26.04.1 LTS
Virtual CPUs	20 (Intel Xeon, as presented by VMware); 4 in the early UTM virtual machine
RAM	48 GB (47 GiB visible to the guest); 7.2 GB, raised to 12 GB, in the early virtual machine
Disk	116 GB virtual disk (the raw dataset is 1.2 GB; the processed arrays (4.2 GB), run records (12 GB) and models (0.2 GB) bring the project to about 16 GB)
Network	Emulated: Mininet hosts and one Open vSwitch switch

Table 3.2: Hardware used for development and evaluation

3.4.2 Software requirements

Software	Version	Purpose
Python	3.14.4	Implementation language
LightGBM	4.7.0	First-stage classifier
PyTorch (CPU)	2.14.0	Autoencoder and DL baseline
Flower (`flwr`)	1.38.0	Federated learning server and clients
scikit-learn	1.9.1	Splits, scaling, metrics
pandas / NumPy / PyArrow	3.0.6 / 2.5.3 / 25.0.1	Data handling
os-ken	4.2.2	OpenFlow 1.3 SDN controller
Mininet	2.3.0	Network emulation
Open vSwitch	3.7.1	Software switch
TShark (Wireshark)	4.6.4	Live feature extraction
tcpreplay	–	Replaying attack captures
Flask	3.1.3	Mitigation REST bridge
Streamlit	1.64.0	Dashboard
pytest	8.4.2	Automated tests
Matplotlib	3.11.2	Figures

Table 3.3: Software used

3.5 Data Requirements

- **Labelled traffic.** A public packet-level dataset with benign traffic and several attack families, so that results can be repeated by others. Edge-IIoTset [1] was chosen (Section 5.2).
- **Raw captures.** The dataset's own `.pcap` files are needed twice: to repair the column-shifted MITM rows, and to replay real attack traffic in Phase B.
- **Connection identity.** The evaluation needs to know which packets belong to the same connection, so that a connection never appears on both sides of a train–test split. Identifiers are used only to group packets and are never given to a model as features.

- **Volume.** 2,219,201 raw rows, which become 1,985,768 unique rows (342,814 connections) with 130 features after cleaning; about 1.2 GB as CSV and a few gigabytes once the arrays and captures are included.
- **Class coverage.** Fifteen classes with a very uneven split (Normal 72.3%, MITM 0.02%); a rare class needs enough rows to be learned and enough connections to be split (MITM is a single connection, see Section 6.2).
- **Privacy.** In deployment the raw traffic would stay on each client; the dataset is split across simulated clients to imitate this.
- **Isolation.** Because the captures contain real attack traffic, all data, training and results are kept inside one virtual machine, and only text (logs and numbers) is copied out.

3.6 Constraints and Limitations

- **One dataset.** All results use Edge-IIoTset, generated on one testbed. Benign traffic in it is only MQTT and VNC, so the service-port feature may over-state how separable web attacks are in a real network.
- **Emulated, not physical.** The network is Mininet and Open vSwitch inside one virtual machine; there are no physical IoT devices and no hardware switch. Clients are processes on one machine, not separate computers, so communication delays are not measured.
- **Batch mitigation.** Phase B classifies a whole capture after its features have been extracted; it is not inline per-packet blocking.
- **Calibration set on the server.** The classifier is trained on labelled data held by the server (5% of the pool); the classifier itself is not federated.
- **Time and compute.** The whole project was done on one virtual machine within one academic year. A full-dataset run takes 17 to 31 minutes, which limited the number of repeat seeds to three for the main experiments and to one for the stealth-attacker, pruning and clipping tests.
- **Cost.** Only free and open-source software and a public dataset were used, so there was no licence cost. A real deployment would need, per site, an edge device able to run the client and an OpenFlow-capable switch or a host running Open vSwitch.

3.7 Assumptions

- The labels of Edge-IIoTset are correct, apart from the problems reported in Section 5.2.1.
- Clients hold different mixes of traffic (non-IID, Dirichlet $\alpha = 0.3$), and most clients are honest.
- At most about 40% of clients are malicious; median-based filtering is assumed to fail at 50% and above.
- The server holds a small, correctly labelled calibration set to start the classifier.
- An analyst can confirm the labels of the few alerts each client surfaces every third round.
- Benign traffic seen by a client during training is representative of its normal traffic later, so the 97th-percentile threshold is meaningful.
- Replaying the dataset's captures is a fair stand-in for live attack traffic in the emulated network.

3.8 Risks and Challenges

Risk	**What happened or could happen**	**How it was handled**
Label leak in the released dataset	A placeholder spelling separated benign from attack with an AUROC of 1.0, so results looked better than they are	Both spellings mapped to one value; a test fails if any single feature exceeds an AUROC of 0.9
Sibling packets in train and test	A random packet split puts packets of one connection on both sides	Split by connection, checked against the row split
Poisoned client updates	30% sign-flip clients drove plain FedAvg's autoencoder loss to about 9×10^{14}	Trust filter, norm clipping and trimmed mean

Risk	**What happened or could happen**	**How it was handled**
Adaptive attacker	A stealth attacker can stay close to the median	Measured and reported; not defended against (Section 6.2)
Memory limits	An early 10-process run ran out of memory on a 7 GB virtual machine	Moved to a larger virtual machine; memory per client measured
Tool version drift	A newer tshark printed booleans differently and silently zeroed one feature on live packets	Live extraction maps both formats back; checked against training rows
Handling real attack traffic	Malicious captures on a personal computer	Everything runs inside one isolated virtual machine; only text leaves it
Weak classes at scale	Autoencoder recall and the trust filter were weaker on the whole dataset than on the 60,000-row pool	Reported openly; cause not yet established

Table 3.4: Risks, challenges and how each was handled

Chapter 4 Designs

This chapter describes how FL-IDS is put together: the overall architecture, the modules and what each is responsible for, the detailed design of the parts that carry the logic, and the data the system reads and writes. The mathematics behind each part is in Chapter 5.

4.1 System Architecture Overview

FL-IDS has three groups of parts (Figure 4.1). The **server** holds a small labelled calibration set, trains the first-stage classifier, runs the trust filter and aggregation, and writes the model bundle. Each **client** (a separate operating-system process) holds its own share of the traffic, trains the autoencoder on it and never uploads raw packets. The **emulated network** (Phase B) loads the bundle, turns replayed attack captures into verdicts and turns verdicts into OpenFlow rules.

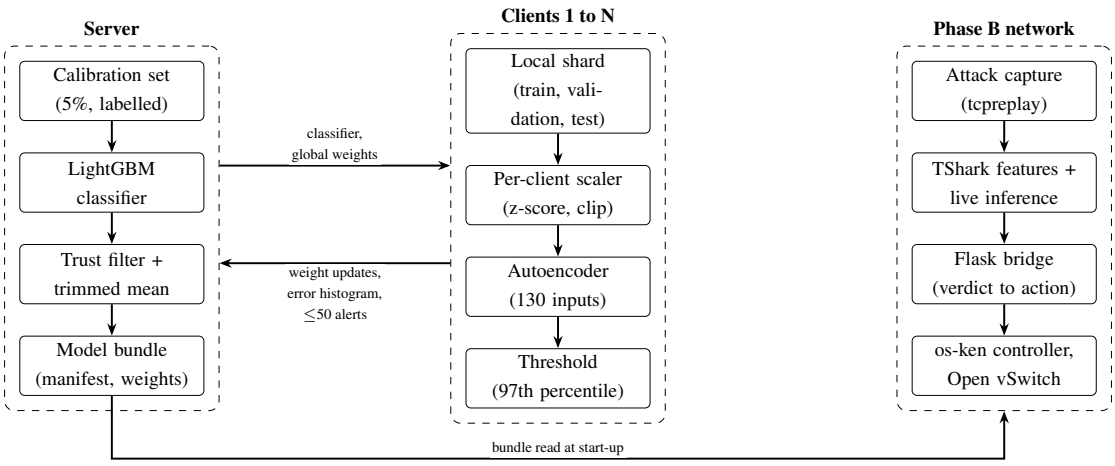


Figure 4.1: Deployment view of FL-IDS: server, clients and the emulated network

Figure 4.2 gives the same design as a block diagram of the two phases.

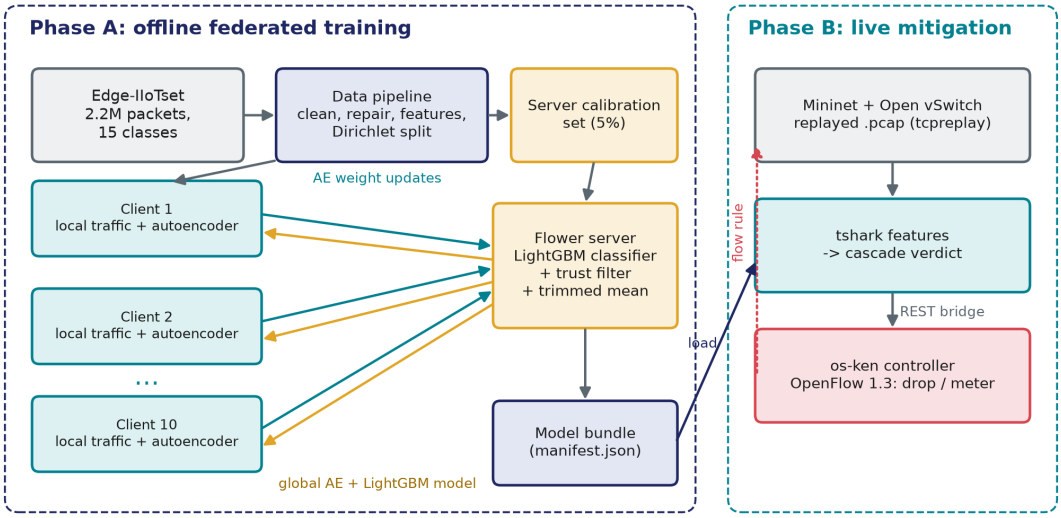


Figure 4.2: High-level design of FL-IDS

Four design decisions shape everything else.

1. **Raw features for the classifier, normalised features for the autoencoder.** Trees split on raw values, so a client's own scaling cannot distort the shared classifier; the autoencoder needs scaled inputs and each client scales with its own statistics.

2. **One decision rule.** The cascade rule exists in a single module and is used by the evaluation code, the dashboard and the SDN path, so the three cannot disagree.

3. **Filtering on deltas.** The trust filter looks at each client's change to the weights, not at the weights, because honest weights differ a lot between non-IID clients but their updates point the same way.

4. **A self-describing bundle.** Phase B rebuilds everything from the bundle's manifest, never from the current configuration, and refuses a bundle whose saved thresholds do not match the saved weights.

4.2 Module/Component Design

The code is a Python package, `fl_ids`, with one sub-package per component. Table 4.1 lists them.

Sub-package	Modules	Responsibility
`data`	`pipeline`, `repair`, `features`, `sampling`, `synthetic`	Cleaning, MITM repair, one-hot encoding, service-port, content and flow features, connection-grouped splitting and the Dirichlet partition; a synthetic generator for tests
`models`	`boosting`, `boosting_update`, `autoencoder`, `cascade`, `dl_classifier`	First-stage classifier and its incremental update, the autoencoder, the shared cascade rule, and the deep-learning baseline
`fl`	`client`, `server`, `strategy`, `launch`, `data_io`	Flower client and server, the custom strategy, starting the processes, loading each client's shard
`robustness`	`trust_filter`, `aggregation`, `attackers`	Cosine/MAD/EMA filter, clipping and trimmed mean, and the sign-flip and stealth test attackers
`sdn`	`topology`, `controller`, `bridge`, `feature_extraction`, `live_inference`, `demo_io`	Mininet topology, os-ken OpenFlow 1.3 controller, Flask bridge, TShark-based feature extraction and per-host scoring
`orchestration`	`phase_a`, `phase_b`, `artifacts`	Phase A and Phase B entry points; saving and loading the model bundle
`eval`	18 modules, including `ablation`, `poisoning_sweep`, `zero_day`, `adaptive_attack`, `pruning`, `metrics`	Every experiment in Chapter 6 and the metrics they share
`dashboard`	`app`, `data`	Streamlit view of training and mitigation state
`utils`	`config`, `journey`, `logging_setup`	Configuration loading, stage timing (the "journey"), logging

Table 4.1: Packages and modules of FL-IDS

Outside the package, `configs/` holds one YAML file per experiment, `tools/` holds the latency benchmark and similar one-off tools, and `tests/` holds the automated tests.

4.3 Detailed Design

4.3.1 One federated round

Figure 5.3 (Chapter 5) shows the arithmetic; Figure 4.3 shows the order of messages. The server never sees a packet: it receives weights, a 50-bin histogram of reconstruction errors, and every third round at most 50 confirmed alerts per surviving client.

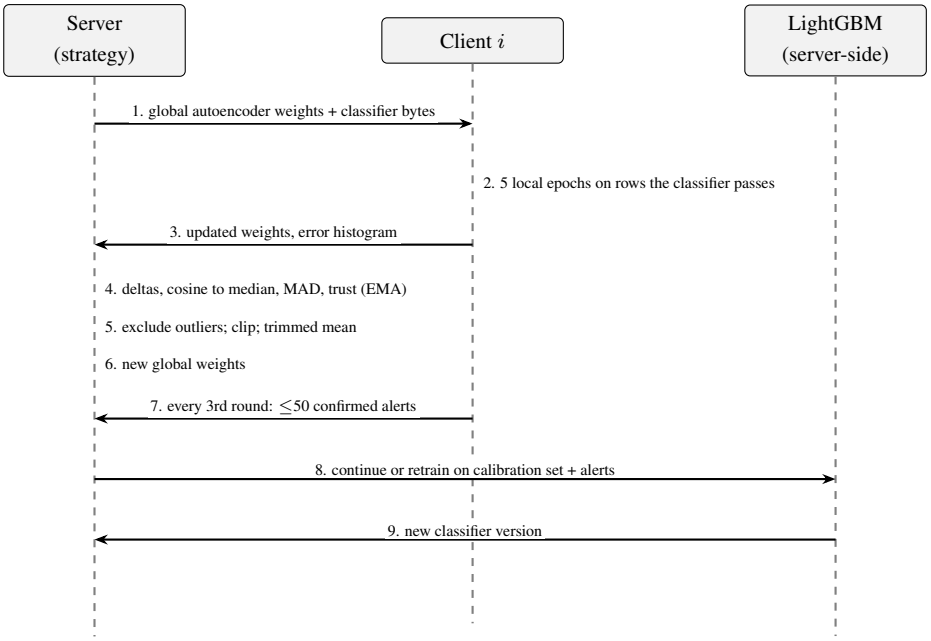


Figure 4.3: Order of messages in one federated round

4.3.2 Cascade decision

1. The LightGBM classifier returns a class and its probability p.
2. If the class is an attack and $p \geq 0.7$ the label is final (the classifier's confidence is reported).
3. Otherwise the autoencoder's reconstruction error e is compared with the client's threshold τ_i; $e > \tau_i$ gives *anomalous*, with confidence $\mathrm{clip}(e/\tau_i - 1, 0, 1)$.
4. Anything else is benign.

The module returns the label, the confidence and the stage that decided, which the bridge and the dashboard reuse. Figure 5.2 (Chapter 5) is the flow chart of this rule.

4.3.3 Trust filter and aggregation

The filter is a pure function of the round's deltas and the previous trust scores: it returns which clients survive, why each excluded client was excluded (`mad_outlier`, `opposes_consensus` or `low_trust`) and the new trust scores. Aggregation is a second pure function that takes the survivors' deltas, clips them and applies the trimmed mean. Keeping both free of Flower types made them testable on small arrays, and `TrustFilteredStrategy` only glues them to Flower's `aggregate_fit`. If no client survives, the round is skipped.

4.3.4 Phase B: from capture to rule

1. Mininet starts one host per federated client on one Open vSwitch switch; the os-ken controller connects over OpenFlow 1.3.
2. `tcpreplay` sends a dataset capture from one host.
3. TShark extracts the dataset's 61 fields; the same feature code as in training derives the 130 features.
4. Each row is scored with that host's scaler and threshold; the most frequent non-benign label is the host's verdict, with its confidence and deciding stage.
5. The Flask bridge receives {device, classification, confidence, stage}. Confidence ≥ 0.85 gives *block*, ≥ 0.5 *rate-limit*, otherwise *allow*.
6. The controller installs `priority=100,ip,nw_src=`*attacker* `actions=drop` (or a meter), and the result is read back from the switch's flow table and saved with the verdict.

Mininet needs root rights; it is started through a single permitted command whose parameters travel through a fixed file, so no other privileged command is exposed.

4.3.5 Classes and interfaces that matter

- **Configuration**: a set of dataclasses (data, boosting, autoencoder, cascade and so on) loaded from YAML, so every value in a run is recorded with the run (`config.yaml` is copied into each run directory).
- **Bundle**: a loader that validates before use (Section 4.4).
- **Cascade**: `flag_attacks(mode, ...)` with the modes `boosting_only`, `autoencoder_only` and `cascade`, which is how the ablation and the latency benchmark call the same code path.
- **Journey**: a stage timer that records how long each step of a run took (loading and cleaning, bootstrap classifier and partition, federated training, threshold calibration, evaluation, saving the bundle).

4.4 Data Design

4.4.1 Dataset and splits

Figure 4.4 shows how the dataset is divided. Rows are grouped into connections first; a connection is never divided.

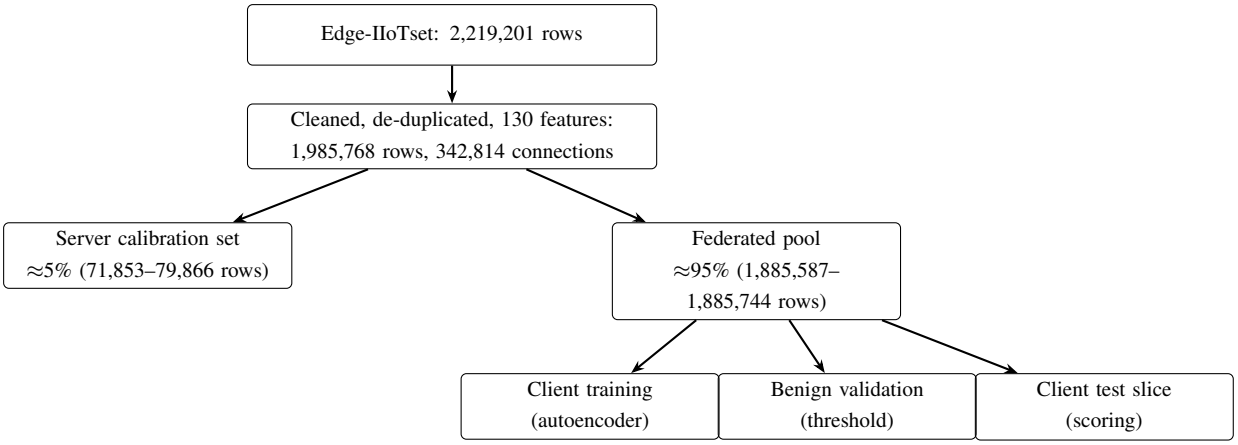


Figure 4.4: How the dataset is divided; every connection goes to exactly one box

The pool is divided among N clients with a Dirichlet($\alpha = 0.3$) label skew, so some clients see almost no benign traffic (up to three of 20 clients had no benign validation rows in the main runs).

4.4.2 Files written by a run

File	Contents
`boosting_model.txt`	The LightGBM model in text form
`autoencoder_weights.npz`	Final global autoencoder weights
`client_normalization.npz`	Each client's mean, scale and threshold
`manifest.json`	Class names, the 130 feature names, input size, autoencoder shape, cascade settings, clip value, thresholds, held-out metrics, training summary (rounds, trust scores, survivors, classifier updates), seed, data set name, creation time and git commit
`journey.json`	Seconds spent in each stage of the run
`runs/.../phase_a/`	Per-client logs and arrays, calibration arrays and the configuration used
`runs/.../phase_b/`	Topology, bridge and controller logs, and `phase_b_result.json` with the verdict, the action and the flow table read back
`results/*.json`	One file per experiment (for example the latency benchmark)

Table 4.2: Files that make up a model bundle and a run record

4.4.3 Record layout

A record is a vector of 130 numeric features. The first 91 are the dataset's own fields after dropping 16 columns and one-hot encoding seven nominal ones; one is the service port, 18 describe payload content and 20 describe the flow the packet belongs to (Section 5.2.3). Class labels are stored as indices into the manifest's `class_names` list, with Normal as the benign class. Identifiers (IP addresses, absolute time, ephemeral

ports) are used only to group packets into flows and connections and are never stored as features.

4.4.4 Bundle validation

On load, Phase B recomputes every client's threshold from the saved weights and checks that it equals the saved threshold and rejects the bundle if they differ. This catches a bundle assembled from two different runs.

Chapter 5 Methodology

FL-IDS runs in two phases. **Phase A** prepares the data, trains the LightGBM classifier, trains the autoencoder over 20 federated rounds with the trust filter, evaluates everything, and saves a model bundle. **Phase B** loads that bundle, classifies replayed attack traffic on an emulated network and turns the verdict into an OpenFlow rule. Figure 4.2 shows the overall design.

5.1 Tools and Techniques

- **LightGBM** [11]: histogram-based gradient-boosted decision trees, used as the first-stage multi-class classifier. Trees work directly on raw feature values, so each client's local normalisation cannot distort the classifier's split thresholds.
- **Autoencoder (PyTorch)**: a dense network $130 \to 32 \to 16 \to 8 \to 16 \to 32 \to 130$ with ReLU activations and a linear output, 9,834 parameters in total (7,299 with the original 91 features). It learns to reconstruct benign traffic, so a high reconstruction error indicates unusual traffic.
- **Flower**: the federated learning framework. A custom strategy, `TrustFiltered-Strategy` (a subclass of FedAvg), implements the trust filter and trimmed-mean aggregation.
- **Cosine similarity, median absolute deviation (MAD) and exponential moving average (EMA)**: the statistics used by the trust filter.
- **Mininet, Open vSwitch, os-ken**: network emulation, the software switch, and the OpenFlow 1.3 controller (os-ken is the maintained fork of Ryu).
- **TShark and tcpreplay**: extracting the dataset's fields from live packets, and replaying captures into the emulated network.
- **Streamlit**: the live dashboard.

5.2 Data Set with Explanation

Edge-IIoTset [1] was generated on a seven-layer IoT/IIoT testbed (cloud, network functions virtualisation, blockchain, fog, SDN, edge and IoT/IIoT perception layers) with more than ten kinds of sensors, including temperature, humidity, water level, pH, soil moisture, heart rate and flame sensors. They communicate over MQTT and Modbus/TCP. The authors' ML-ready file, `DNN-EdgeIIoT-dataset.csv`, has 2,219,201 rows and 63 columns: 61 packet-level fields extracted with tshark, plus `Attack_label` and `Attack_type`. The 24 per-device and per-attack `.pcap` captures are also used, for Phase B replay and for repairing corrupted rows.

There are 15 classes: Normal and 14 attacks in five groups. Table 5.1 gives the class sizes after cleaning (current code, before the new features are added).

Group	**Class**	**Rows**	**%**
–	Normal	1,363,998	71.42
DoS/DDoS	DDoS_UDP	121,567	6.37
	DDoS_ICMP	67,939	3.56
	DDoS_TCP	50,062	2.62
	DDoS_HTTP	48,544	2.54
Injection	SQL_injection	50,826	2.66
	Uploading	36,807	1.93
	XSS	15,066	0.79
Information gathering	Vulnerability_scanner	50,026	2.62
	Port_Scanning	19,977	1.05
	Fingerprinting	853	0.04
Malware	Password	49,933	2.61
	Backdoor	24,026	1.26
	Ransomware	9,689	0.51
Man-in-the-middle	MITM	406	0.02
Total		**1,909,719**	

Table 5.1: Class distribution after cleaning (91 base features)

5.2.1 Data-quality problems found and fixed

1. **Placeholder label leak.** In the categorical columns, the "field does not apply" value is written as `0` in some source captures and `0.0` in others, and

the spelling follows the label. In `mqtt.topic`, `mqtt.conack.flags`, `mqtt.protoname` and `dns.qry.name.len`, every Normal row uses `0` and every attack row `0.0`. After one-hot encoding, the single column `mqtt.topic_0.0` separates benign from attack with an AUROC of 1.0. Both spellings are mapped to `0` before encoding, after which the strongest single feature has an AUROC of about 0.71. A test guards that no single feature exceeds 0.9. We found this independently, but Galal [13] reported the same serialisation artefact first, so it is not claimed as a contribution of this project.

2. **Column-shifted MITM rows.** In the authors' MITM rows, values sit under the wrong column names (for example, their `tcp.seq` is really `udp.time_delta`, with 1.000 agreement). These rows are replaced by re-extracting the authors' own MITM capture with the same field list (1,214 rows replaced by 1,229). Galal [13] keeps these rows as released and does not report the shift; we found no other report of it.

3. **TShark version drift.** Newer tshark prints booleans as `True`/`False` and shortens hexadecimal values. Without a fix, this silently zeroed `tcp.flags.ack` on every live packet in Phase B. Live extraction maps both back to the authors' format.

5.2.2 Preprocessing

Sixteen columns are dropped, following the dataset authors' recipe: identifiers, timestamps, raw payloads and ports, plus `Attack_label`, which is fully determined by `Attack_type`. Seven nominal columns are one-hot encoded, giving **91 base features**. Exact duplicate rows are removed (309,497 rows), leaving 1,909,719 rows.

5.2.3 New features for the classifier

Profiling the dropped columns explained why the application-layer attacks were hard to classify. In the SQL_injection, XSS, Uploading and Password classes, only 5–17% of packets carry any HTTP content. The rest are content-free TCP handshake and acknowledgement packets to port 80 on the same web server, which look the same whichever attack produced them. Three groups of features were therefore added. IP addresses, absolute time and ephemeral client ports are *never* used as features, because each of them identifies the capture file and hence the label.

1. **Service port (1 feature):** the server side of a connection, $\min(\text{ports} < 32768)$, or -1 if there is none (for example ICMP or ARP).

2. **Content features (18):** decoded payload length, byte entropy, printable fraction, HTTP request flag, HTTP status code, header count, `X-` header count, URI length, parameter count, percent-encoding count, file-data length, special-character count, and counts of six generic attack-signature keyword groups (SQL, script/XSS, upload, authentication, traversal/command, scanner tool names).

3. **Flow features (20):** packets are grouped into flows by (IP pair, protocol, port pair) in capture order. A new flow starts after a 60 s idle gap or once a flow has been active for 120 s, CICFlowMeter's default active timeout [14]. Every packet in a flow receives the flow's packet count, duration, packet rate, payload sum/maximum/non-zero share, HTTP request count, maximum status code, SYN, FIN and RST counts, maximum entropy, maximum URI length and keyword totals. IP addresses and times are used only to group packets, never as features.

With all three groups there are 130 features and 1,985,768 unique rows.

Class split of the final data. After the new features are added and duplicates are removed, 1,985,768 unique rows remain; Table 5.2 gives the class sizes (the 342,814 connections they form are what the splits keep whole). Table 5.1 above was counted on the 91 base features before the last cleaning step and is kept to show what changed.

Group	**Class**	**Rows**	**%**
–	Normal	1,436,597	72.34
DoS/DDoS	DDoS_UDP	121,567	6.12
DoS/DDoS	DDoS_ICMP	68,758	3.46
DoS/DDoS	DDoS_TCP	50,062	2.52
DoS/DDoS	DDoS_HTTP	49,768	2.51
Injection	SQL_injection	50,981	2.57
Injection	Uploading	37,243	1.88
Injection	XSS	15,172	0.76
Information gathering	Vulnerability_scanner	50,044	2.52
Information gathering	Port_Scanning	20,213	1.02
Information gathering	Fingerprinting	872	0.04
Malware	Password	50,033	2.52
Malware	Backdoor	24,167	1.22
Malware	Ransomware	9,885	0.50
Man-in-the-middle	MITM	406	0.02
Total		**1,985,768**	**100.00**

Table 5.2: Class distribution of the final data (130 features)

5.3 Flowchart of Methodology from Start to End

Figure 5.1 shows the complete sequence of steps, from loading the dataset to verifying the mitigation rule.

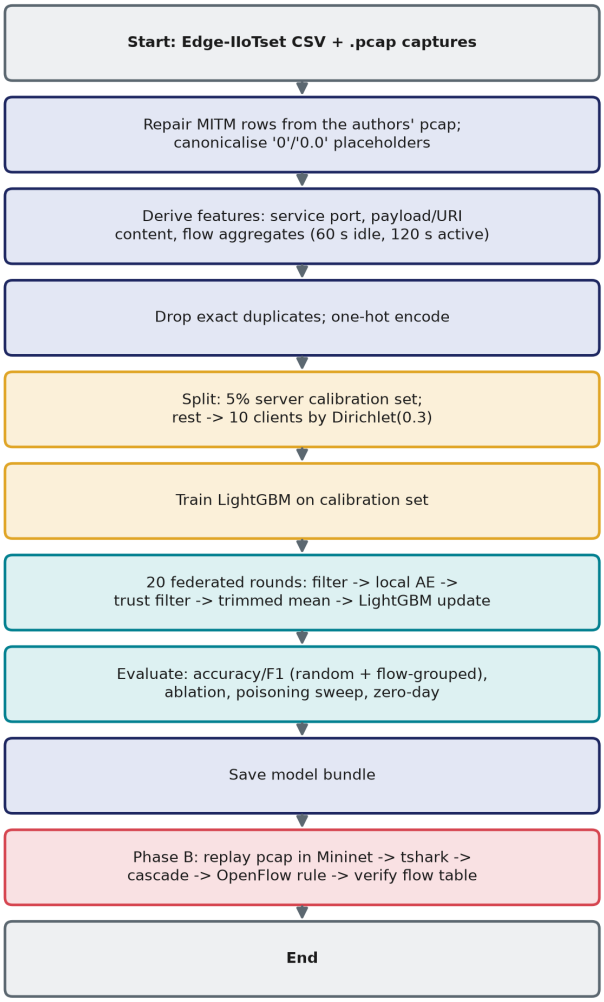


Figure 5.1: Flow chart of the methodology

5.4 Explanation of Methodology in Detail with Equations and Formulae

5.4.1 How much data each part uses

Every packet of the dataset is cleaned and given the new features (2,219,216 rows, 1,985,768 unique after de-duplication). How many of them each model trains on differs, and Table 5.3 lists it. The classifier that meets the accuracy target is trained on most of the data. The federated system was first run on a pool capped at 60,000 rows, to keep the roughly 150 evaluation runs practical; the main federated results (Section 6.1.2) use the whole pool.

Part	Trained on	Evaluated on
Final classifier (accuracy target)	1,429,825 rows (72%), plus 158,870 for early stopping (8%)	397,073 unseen rows (20%); whole connections held out
Same classifier, 5% share only	89,359 rows, plus 9,929 for early stopping	the same 397,073 rows
Federated: server bootstrap classifier	79,430 rows (calibration set)	19,858 rows held back
Federated, earlier 60k pool: client autoencoders	45,474 rows across 10 clients (736 to 11,549 each), plus 5,530 benign rows for thresholds	8,996 client test rows
Federated, full pool: server bootstrap classifier	71,853 to 79,866 rows (calibration set, whole connections)	a part held back for scoring boosting versions
Federated, full pool: client autoencoders	the 1,885,587 to 1,885,744 pool rows across 10 or 20 clients, split by connection (training, benign-validation and test slices)	283,090 to 295,566 client test rows

Table 5.3: Training and evaluation rows for each part (130 features, seed 42)

The 5%-only row shows that the federated bootstrap loses little from its smaller training set: 99.7% accuracy and 0.979 macro F1 on the same 397,073 test rows, against 0.983 for the full classifier.

5.4.2 Non-IID partition and per-client normalisation

A stratified 5% of the rows is held on the server as the calibration set. The remaining federated pool (all of it in the main runs; subsampled to 60,000 rows in the earlier comparison runs) is split across $N = 10$ clients. For each class k a proportion vector is drawn,

$$\mathbf{p}_k \sim \mathrm{Dir}(\alpha \mathbf{1}_N), \qquad \alpha = 0.3, \tag{5.1}$$

and the class's rows are divided in those proportions [12]. Smaller α gives more skew. In the full-pool runs the unit that is divided is the connection, not the row: each connection goes to one client, and within a client it goes to exactly one of the training, benign-validation and test slices, so no test packet has a sibling from the same connection in training. Each client keeps 15% as a test slice and 15% of its benign remainder

as a benign validation slice. For the autoencoder only, client i standardises features with its own training mean μ_i and standard deviation σ_i, clipped at ± 10:

$$\tilde{x} = \mathrm{clip}\left(\frac{x - \mu_i}{\sigma_i},\ -10,\ 10\right). \tag{5.2}$$

The clip is needed because a counter feature can barely vary inside one skewed shard. Without it, one client's benign validation loss stayed near 400 for 20 rounds; with it, the loss fell to 0.0146.

5.4.3 Stage 1: LightGBM classifier

LightGBM builds an additive model of T trees per class, $F_k(\mathbf{x}) = \sum_{t=1}^{T} f_{t,k}(\mathbf{x})$, with class probabilities

$$P(y = k \mid \mathbf{x}) = \frac{e^{F_k(\mathbf{x})}}{\sum_{j=1}^{15} e^{F_j(\mathbf{x})}}, \tag{5.3}$$

trained by minimising the weighted multi-class log loss

$$\mathcal{L} = -\frac{1}{\sum_n w_n} \sum_n w_n \log P(y = y_n \mid \mathbf{x}_n) + \lambda_2 \sum_t \|\mathbf{v}_t\|^2, \tag{5.4}$$

where $\mathbf{v}_t$ are leaf values and $\lambda_2 = 1$. The sample weight w_n is a square-root inverse class frequency, recomputed for every training set (the bootstrap and each incremental update),

$$w_n = \sqrt{\frac{\max_j N_j}{N_{y_n}}}, \tag{5.5}$$

so a rare class such as Fingerprinting counts more without being blown up the way full inverse-frequency weights would. In the federated pipeline the classifier uses 300 rounds, learning rate 0.05 and 31 leaves, trained deterministically on one thread so runs are reproducible. The stand-alone accuracy experiments of Section 6.1.1 used 63 leaves, learning rate 0.1, feature and bagging fractions of 0.8, and early stopping after 50 rounds without improvement on a 10% validation split.

5.4.4 Stage 2: autoencoder and threshold

Each client trains the autoencoder g_θ on the rows the current classifier passes (predicted Normal, or an attack with probability below 0.7). It minimises the mean squared reconstruction error

$$e(\mathbf{x}) = \frac{1}{d} \sum_{j=1}^{d} \left(\tilde{x}_j - g_\theta(\tilde{\mathbf{x}})_j\right)^2 \tag{5.6}$$

with Adam (learning rate 10^{-3}, 5 local epochs, batch size 64). The client's threshold is recomputed every round as the 97th percentile of e on its benign validation slice, $\tau_i = Q_{0.97}\{e(\mathbf{x}) : \mathbf{x} \in V_i\}$.

5.4.5 Cascade decision rule

Figure 5.2 shows the shared decision rule. With $\hat{k} = \arg\max_k P(k \mid \mathbf{x})$ and $p = P(\hat{k} \mid \mathbf{x})$:

$$\text{label}(\mathbf{x}) = \begin{cases} \hat{k} & \text{if } \hat{k} \neq \text{Normal and } p \geq 0.7, \\ \text{anomalous} & \text{else if } e(\mathbf{x}) > \tau_i, \\ \text{benign} & \text{otherwise,} \end{cases} \tag{5.7}$$

with confidence p for a known attack and $\text{clip}(e/\tau_i - 1,\ 0,\ 1)$ for an anomaly.

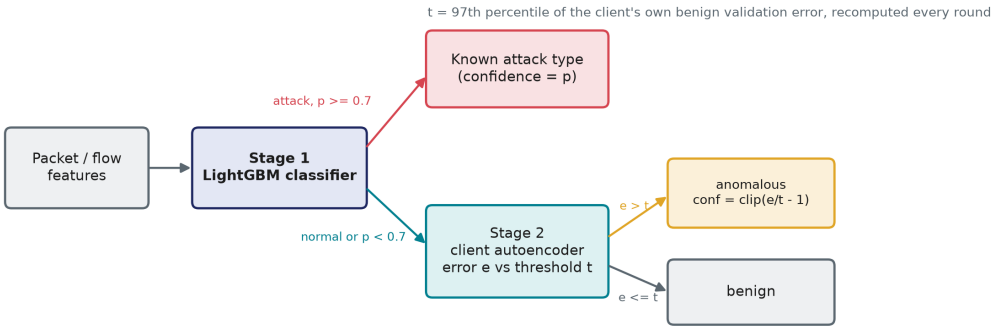


Figure 5.2: Two-stage cascade decision rule

5.4.6 Trust filter and robust aggregation

Let $\mathbf{w}^{(r)}$ be the global weights at the start of round r and $\mathbf{w}_i$ client i's weights after local training. The filter works on *deltas* $\Delta_i = \mathbf{w}_i - \mathbf{w}^{(r)}$. The reference direction is the coordinate-wise median $\mathbf{m} = \text{median}_i(\Delta_i)$, and each client's similarity is

$$s_i = \frac{\Delta_i \cdot \mathbf{m}}{\|\Delta_i\|\,\|\mathbf{m}\|}. \tag{5.8}$$

Trust is a running average, starting at $T_i = 1$:

$$T_i \leftarrow (1-\beta)\,T_i + \beta\,\frac{s_i + 1}{2}, \qquad \beta = 0.3. \tag{5.9}$$

Client i is excluded in round r if any of these holds:

$$s_i < \tilde{s} - 3\,\text{MAD}(s) \quad \text{or} \quad s_i < 0 \quad \text{or} \quad T_i < 0.5, \tag{5.10}$$

where $\tilde{s}$ is the median similarity. Surviving deltas are clipped to twice the median norm ν,

$$\Delta_i' = \Delta_i \cdot \min\left(1,\ \frac{2\nu}{\|\Delta_i\|}\right), \tag{5.11}$$

and combined coordinate-wise with a 15% trimmed mean, which drops the largest and smallest $\lfloor 0.15\, n \rfloor$ values in each coordinate:

$$\mathbf{w}^{(r+1)} = \mathbf{w}^{(r)} + \mathrm{TrimMean}_{0.15}\big(\{\Delta_i'\}\big). \tag{5.12}$$

The test attacker trains honestly and then sends $-5\Delta_i$ (sign flip, amplified five times), while claiming the largest client's example count. Figure 5.3 summarises one round.

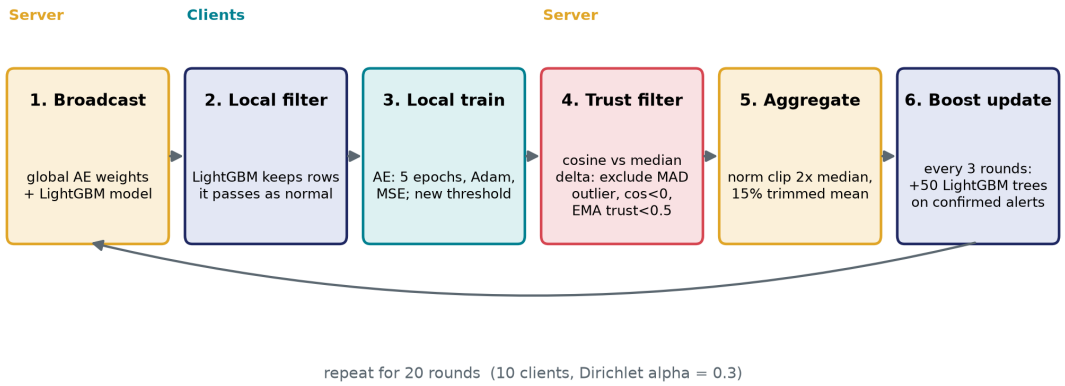


Figure 5.3: One federated round

5.4.7 Incremental classifier update

Every 3 rounds, each client that survived the filter surfaces up to 50 rows its autoencoder flagged, with analyst-confirmed labels. The server continues training LightGBM with 50 more trees on the calibration set plus all collected alerts, or retrains from scratch if the alerts contain a class the model has never seen. This relaxes the "no raw traffic leaves a client" rule for these capped, confirmed alerts only, and is reported as a privacy trade-off.

5.4.8 SDN mitigation

In Phase B, Mininet creates one host per federated client on one Open vSwitch switch, controlled by os-ken over OpenFlow 1.3. A real attack capture is replayed with tcpreplay. TShark extracts all 61 of the dataset's fields, the service-port, content and flow features are derived from the capture with the same code as in training, each row is

scored with that host's own scaler and threshold, and the most frequent non-benign label becomes the device verdict. A Flask bridge maps the verdict to an action: *block* (a drop rule on the source IP) if confidence ≥ 0.85, *rate-limit* (an OpenFlow meter) if ≥ 0.5, otherwise allow. The installed rule is then read back from the switch's flow table to confirm it.

5.4.9 Evaluation metrics

With TP_k, FP_k, FN_k counted per class k over K classes:

$$\text{Accuracy} = \frac{\#\text{correct}}{\#\text{test rows}}, \qquad P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}, \tag{5.13}$$

$$F1_k = \frac{2P_kR_k}{P_k + R_k}, \qquad \text{Macro F1} = \frac{1}{K}\sum_k F1_k. \tag{5.14}$$

Because 71% of traffic is benign, accuracy alone hides poor results on rare attacks, so macro F1 is reported next to it. A class with no rows in a test split (MITM is a single flow) is left out of that split's macro average. Two splits are used: a **stratified random 80/20 split** of rows, and a stricter **flow-grouped 80/20 split** (stratified group k-fold), in which every flow lies entirely in training or entirely in test, so the model cannot score well by memorising connections. The **ambiguity ceiling**, the accuracy of an oracle that gives every distinct feature vector its majority label, shows how separable the classes are with a given feature set. For the federated system, attack macro-recall, benign false-positive rate, zero-day macro-recall (leave one attack class out of all training data, then score it) and malicious-client survival rate are also reported.

Chapter 6 Results and Discussion

6.1 Presentation of Results

All numbers in this chapter come from logged runs on the hardware in Section 3.4.1. Unless stated otherwise, the seed is 42. The chapter is ordered from the strongest evidence to the weakest: the stand-alone classifier (Section 6.1.1), the whole federated system on the whole dataset split by connection (Section 6.1.2), how long training and a decision take (Section 6.1.3), the comparison with a deep-learning classifier, and finally the earlier experiments on a 60,000-row pool, which are kept for comparison (Section 6.1.5).

Scale of the results. The earlier experiments use a 60,000-row federated pool split row by row, to keep roughly 150 evaluation runs practical. The full-dataset experiments repeat the federated system on all 1,985,768 usable rows, split by connection, and give the headline federated results.

6.1.1 Classifier accuracy

The project requirement was at least 95% accuracy and at least 0.95 macro F1. The earlier classifier comparison, on a 500,000-row class-balanced sample of the base features, gave 0.891 accuracy and 0.887 macro F1. On all 1.9 million rows with the natural class distribution, the same base features give 0.964 accuracy but still only 0.887 macro F1, because the application-layer attacks are confused with each other.

The final classifier is trained on 1,429,825 packets and tested on 397,073 packets from connections it never saw (Table 5.3).

6.1.1.1 Ambiguity ceiling

Before tuning the model, we measured how many rows are impossible to separate with each feature set (Table 6.1). With the 91 base features, 2.31% of rows share an identical feature vector with a row of a different class, which caps macro recall at 0.969 for *any* per-packet classifier. The new features bring the ambiguous share down to 0.01%.

Feature set	**Features**	**Ambiguous**	**Ceiling acc.**	**Ceiling macro rec.**
Base	91	2.31%	0.9875	0.9692
+ service port	92	0.29%	0.9985	0.9940
+ content + flow	130	0.01%	0.9999	0.9997

Table 6.1: Ambiguity ceiling per feature set (all rows, after de-duplication)

6.1.1.2 Effect of each feature group

Table 6.2 and Figure 6.1 add the feature groups one at a time, on the strict flow-grouped split. The service port mainly fixes Normal versus attack traffic. The content features add little on their own, because only a small share of packets carries content. The flow features give the largest jump, because they pass a connection's content and behaviour on to its content-free packets. Class weighting then stabilises the rare Fingerprinting class.

	Random split		**Flow-grouped split**	
Features	**Accuracy**	**Macro F1**	**Accuracy**	**Macro F1**
Base (91)	0.9638	0.8868	0.9637	0.8892
+ service port (92)	0.9802	0.9245	0.9803	0.9236
+ content (110)	0.9802	0.9268	0.9802	0.9232
+ flow (130)	0.9977	0.9879	0.9975	0.9808
+ class weights (final)	**0.9976**	**0.9867**	**0.9972**	**0.9828**

Table 6.2: Accuracy and macro F1 as feature groups are added (LightGBM, all rows, 80/20)

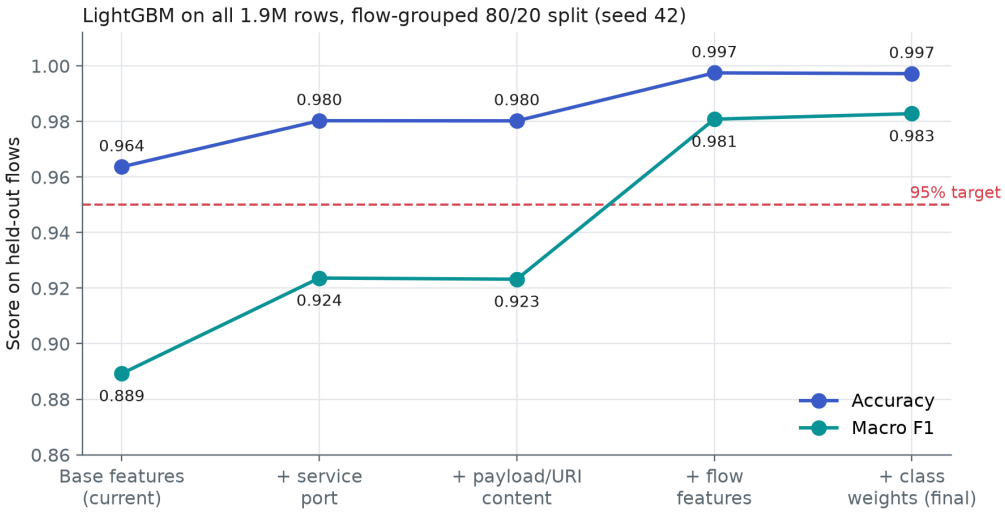


Figure 6.1: Accuracy and macro F1 on held-out flows as each feature group is added

6.1.1.3 Robustness across seeds, splits and training size

Table 6.3 repeats the final configuration with three seeds on both splits. Accuracy stays between 0.9970 and 0.9977, and macro F1 between 0.9828 and 0.9868. Without class weights, the flow-grouped macro F1 was less stable (0.9808, 0.9560, 0.9521), because the Fingerprinting score depended on which of its 156 flows landed in test. Weighting removes that sensitivity.

Split / seed	**42**	**43**	**44**
Random: accuracy	0.9976	0.9977	0.9976
Random: macro F1	0.9867	0.9868	0.9866
Flow-grouped: accuracy	0.9972	0.9971	0.9970
Flow-grouped: macro F1	0.9828	0.9835	0.9851

Table 6.3: Final classifier across seeds

Two further checks were run. **(a) Same sampling as before:** on a 500,000-row capped sample drawn the same way as in the earlier boosting-versus-DL comparison, and with the same training settings for both feature sets, accuracy rises from 0.8902 to 0.9919 and macro F1 from 0.8827 to 0.9900. **(b) Calibration-set size:** in FL-IDS the server trains LightGBM on only a 5% calibration set. Trained on 5% of the rows and tested on held-out flows, the new features give 0.9968 accuracy and 0.9791 macro F1, against 0.9624 and 0.8807 for the base features. The gain therefore carries over to the classifier as the federated system actually trains it.

6.1.1.4 Per-class results

Figure 6.2 compares per-class F1 before and after. The biggest gains are on exactly the classes that were weak: Uploading (0.76 to 1.00), DDoS_HTTP (0.76 to 0.97), SQL_injection (0.80 to 1.00), Password (0.84 to 0.97), XSS (0.85 to 0.98) and Fingerprinting (0.68 to 0.93). Figure 6.3 shows the remaining confusion: Password and DDoS_HTTP traffic (both HTTP requests to the same server) and Ransomware versus Backdoor (both on port 4321). MITM is a single ARP flow, so it cannot be scored on the flow-grouped split; on the random split its F1 is 0.994.

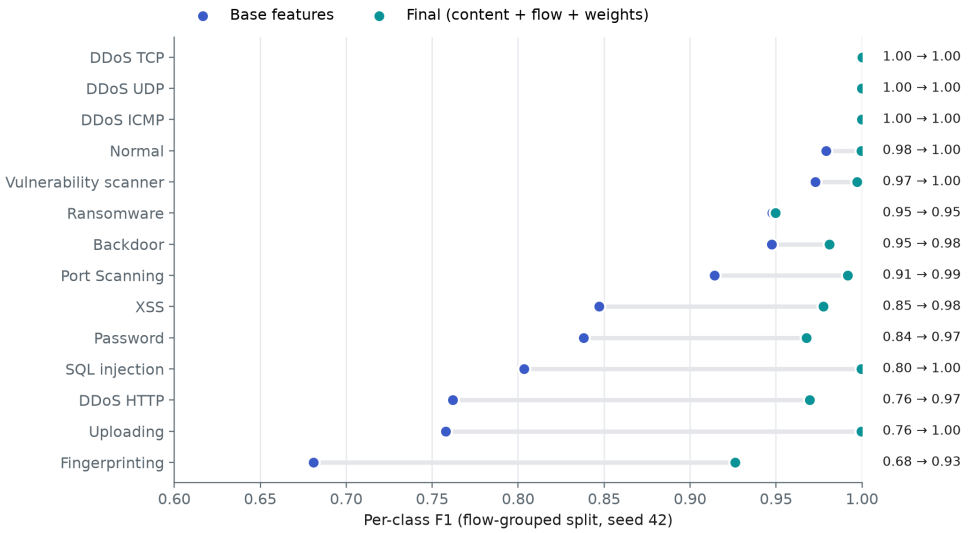


Figure 6.2: Per-class F1, base features vs. final classifier (flow-grouped split)

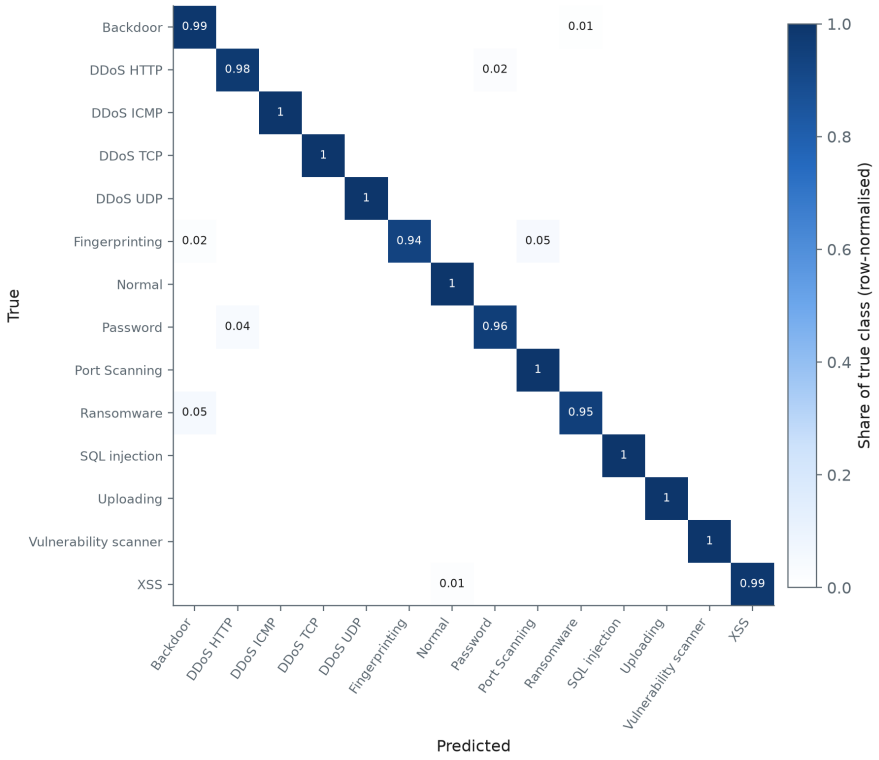


Figure 6.3: Row-normalised confusion matrix of the final classifier (flow-grouped split)

6.1.1.5 Discussion and caveats

- **Why a flow-grouped split matters.** A random split of packets puts packets of the same connection in both training and test sets. Reporting the stricter split shows that the improvement does not come from memorising connections.
- **The service port is partly a testbed effect.** In Edge-IIoTset, benign traffic is MQTT (port 1883) and VNC (5900) and never HTTP. In a real network with benign web traffic, the port alone would separate much less.
- **Flow features are computed over the whole flow record** (up to 120 s), the same way as CICFlowMeter. The classifier therefore labels flow records rather than making a decision on the first packet.
- The same features are used by the whole federated pipeline and by Phase B. The following sections report the system-level results with them, next to the original 91-feature results.

6.1.2 The whole system on the full dataset, split by connection

All of the federated results in the earlier sections of this chapter use a 60,000-row pool split row by row. This section repeats the federated experiments on all 1,985,768 usable

rows. The server keeps a labelled calibration set (71,853 to 79,866 rows, about 5%, depending on the seed) to bootstrap the classifier; the other 1,885,587 to 1,885,744 rows are split across 10 or 20 clients, whose test slices hold 283,090 to 295,566 rows (about 9,000 before). Packets of one connection share their flow-level features, so a row-level split puts near-identical siblings on both sides of a test. Here every connection stays whole: in the calibration set, in exactly one client, and in exactly one of that client's training, benign-validation and test slices. MITM is a single connection, so it cannot be split; all of it goes to the server's calibration set (the classifier learns it) and it is scored by the zero-day experiment and the live replay only.

A leak check first. At the old 60,000-row scale, splitting by connection did not lower known-attack recall (the classifier alone caught 0.9962 of attack packets, against 0.9968 with the row split), so the earlier federated recall numbers were not inflated by sibling packets.

Real runs. Table 6.4 lists six real multi-process runs (one server and 10 or 20 client processes, 20 rounds, three seeds). Known attacks are caught as before: micro recall 0.9956 to 0.9989 and classifier-alone micro recall 0.9954 to 0.9981 (0.9939 to 0.9980 in the earlier runs; Table 6.5). False alarms need two numbers. Counted per packet, 2.5% to 16.3% of normal test packets are flagged (earlier 2.8% to 4.0%), but counted per connection the rate is 0.34% to 1.04%, and the median client flags 2.9% to 3.2% of its packets in all six runs. The packet figure is dominated by a few very long normal connections (7,000 to 11,800 packets each): in five of the six runs, 80% to 94% of the false-alarm packets come from the ten largest connections. The false-alarm rate should therefore be quoted both ways.

Run	Phase A s	Attack macro-recall	Attack micro recall	Classifier-only micro recall	Packet-level false alarms	Connection-l
10 clients, seed 42	1844	0.9958	0.9974	0.9970	0.1275	0.
10 clients, seed 43	1397	0.9947	0.9978	0.9971	0.0884	0.
10 clients, seed 44	1754	0.9939	0.9956	0.9954	0.0248	0.
20 clients, seed 42	1260	0.9981	0.9989	0.9981	0.0588	0.
20 clients, seed 43	1063	0.9970	0.9980	0.9974	0.1628	0.
20 clients, seed 44	1225	0.9972	0.9978	0.9972	0.1089	0.

Table 6.4: Real multi-process Phase A on the full pool (1.89M rows), split by connection, 20 rounds, no attackers

Setting	Attack macro-recall	Attack micro recall	Classifier-only micro recall	Benign FPR (packets)
10 clients, seed 42: 60k pool, split by row	0.9990	0.9992	0.9968	0.0340
10 clients, seed 42: full pool, split by connection	0.9958	0.9974	0.9970	0.1275
10 clients, seed 42: 60k pool, split by connection	1.0000	1.0000	0.9962	0.0443
10 clients, seed 43: 60k pool, split by row	0.9985	0.9992	0.9976	0.0311
10 clients, seed 43: full pool, split by connection	0.9947	0.9978	0.9971	0.0884
10 clients, seed 44: 60k pool, split by row	0.9962	0.9960	0.9939	0.0284
10 clients, seed 44: full pool, split by connection	0.9939	0.9956	0.9954	0.0248
20 clients, seed 42: 60k pool, split by row	0.9784	0.9964	0.9952	0.0362
20 clients, seed 42: full pool, split by connection	0.9981	0.9989	0.9981	0.0588
20 clients, seed 43: 60k pool, split by row	0.9993	0.9992	0.9980	0.0284
20 clients, seed 43: full pool, split by connection	0.9970	0.9980	0.9974	0.1628
20 clients, seed 44: 60k pool, split by row	0.9991	0.9988	0.9968	0.0396
20 clients, seed 44: full pool, split by connection	0.9972	0.9978	0.9972	0.1089

Table 6.5: The same runs at the earlier scale and now (macro-recall includes only attack types present in the test slices)

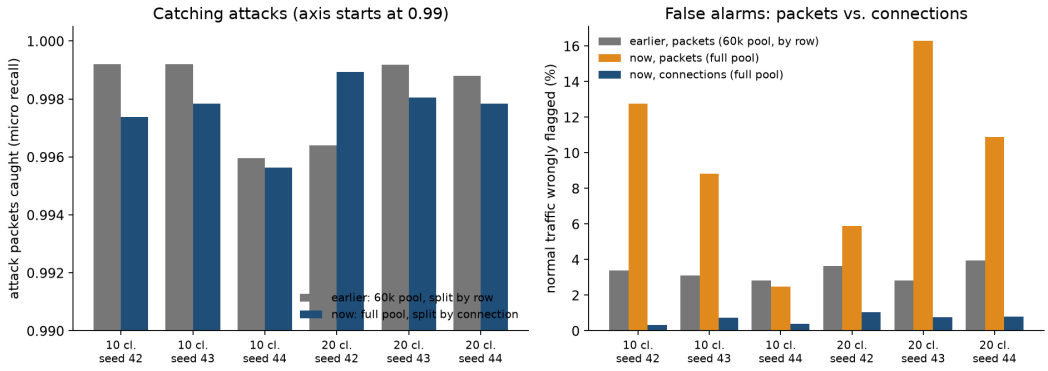


Figure 6.4: Earlier (60k pool, split by row) and full-pool (split by connection) results for the six runs

Class	Test rows	Classifier alone	Autoencoder alone	Cascade
Backdoor	2263	0.9885	0.0367	0.9929
DDoS_HTTP	6915	0.9899	0.4807	0.9899
DDoS_ICMP	9520	1.0000	0.3171	1.0000
DDoS_TCP	6717	1.0000	0.9394	1.0000
DDoS_UDP	17755	0.9999	0.3063	1.0000
Fingerprinting	129	0.9845	0.0698	0.9922
Password	7795	0.9913	0.2517	0.9914
Port_Scanning	2929	1.0000	0.6842	1.0000
Ransomware	691	0.9841	0.4877	0.9870
SQL_injection	7228	0.9990	0.8944	0.9996
Uploading	5567	0.9998	0.5493	0.9998
Vulnerability_scanner	7216	0.9946	0.5006	0.9961
XSS	2274	0.9960	0.4714	0.9969

Table 6.6: Recall per attack type, 10 clients, seed 42, full pool, by stage (held-out test slices)

Poisoning and ablation. Known-attack recall stays between 0.985 and 0.997 at every attacker fraction up to 50% (Table 6.7). The trust filter lets more attackers through than at 60,000 rows: attacker survival at 30% is 0.50, 0.55 and 0.62 (earlier 0.12, 0.12 and 0.27), and at 50% it is 0.64, 1.00 and 0.99; why is not established. Robust aggregation still protects the autoencoder: with 30% attackers its recall alone is 0.490, 0.373 and 0.245 for the full pipeline, against 0.013, 0.002 and 0.092 for plain FedAvg (Table 6.8). The ranges are wide and three seeds support no confidence intervals. The ablation's full-pipeline row is the sweep's 30% point.

Attackers	Attack macro-recall	Autoencoder-alone recall	Attacker survival
0%	0.996 / 0.996 / 0.994	0.457 / 0.492 / 0.221	n/a / n/a / n/a
20%	0.995 / 0.996 / 0.994	0.466 / 0.483 / 0.258	0.30 / 0.40 / 0.23
30%	0.995 / 0.997 / 0.995	0.490 / 0.373 / 0.245	0.50 / 0.55 / 0.62
40%	0.994 / 0.994 / 0.992	0.297 / 0.139 / 0.074	0.60 / 0.64 / 0.61
50%	0.995 / 0.990 / 0.985	0.249 / 0.000 / 0.000	0.64 / 1.00 / 0.99

Table 6.7: Poisoning sweep, full pool

Variant	Attack macro-recall	Autoencoder-alone recall	Benign FPR
full_pipeline	0.995 / 0.997 / 0.995	0.490 / 0.373 / 0.245	0.1112 / 0.0653 / 0.0075
autoencoder_only	0.105 / 0.073 / 0.051	0.105 / 0.073 / 0.051	0.0473 / 0.0411 / 0.0445
boosting_only	0.990 / 0.989 / 0.982	n/a / n/a / n/a	0.0001 / 0.0001 / 0.0003
plain_fedavg	0.992 / 0.994 / 0.990	0.013 / 0.002 / 0.092	0.0947 / 0.0301 / 0.0301
trimmed_mean_only	0.995 / 0.995 / 0.993	0.205 / 0.000 / 0.025	0.1070 / 0.0261 / 0.0299

Table 6.8: Ablation at 30% sign-flip attackers, full pool, seeds 42 / 43 / 44

Zero-day detection. The cascade detects 0.910, 0.944 and 0.895 of an unseen attack type on average (earlier 0.943, 0.952 and 0.952); every row of each held-out class is scored (Tables 6.9 and 6.10, Figure 6.5). Uploading is the weakest (0.527 to 0.665), followed by MITM (0.751 to 1.000) and, depending on the seed, Backdoor and Vulnerability_scanner.

Stage	Mean detection rate
Classifier alone	0.735 / 0.757 / 0.723 (classes: 14, 14, 14)
Autoencoder alone	0.681 / 0.728 / 0.590 (classes: 14, 14, 14)
Cascade	0.910 / 0.944 / 0.895 (classes: 14, 14, 14)

Table 6.9: Zero-day (leave one attack class out), full pool: mean over the held-out classes

Held-out class	Rows scored	Cascade detection	Classifier alone	Autoencoder alone
Backdoor	24167	0.743 / 0.998 / 0.998	0.486 / 0.996 / 0.997	0.493 / 0.550 / 0.810
DDoS_HTTP	49768	0.879 / 0.932 / 0.885	0.695 / 0.764 / 0.719	0.572 / 0.698 / 0.560
DDoS_ICMP	68758	1.000 / 1.000 / 1.000	1.000 / 1.000 / 0.999	0.786 / 0.802 / 0.800
DDoS_TCP	50062	0.933 / 0.953 / 0.921	0.865 / 0.921 / 0.879	0.640 / 0.686 / 0.615
DDoS_UDP	121567	0.905 / 0.803 / 0.866	0.416 / 0.021 / 0.334	0.849 / 0.799 / 0.799
Fingerprinting	872	0.998 / 0.998 / 0.997	0.995 / 0.994 / 0.994	0.826 / 0.747 / 0.492
MITM	406	0.803 / 1.000 / 0.751	0.148 / 0.200 / 0.005	0.764 / 1.000 / 0.751
Password	50033	0.932 / 0.935 / 0.880	0.872 / 0.865 / 0.813	0.505 / 0.407 / 0.307
Port_Scanning	20213	0.999 / 0.993 / 0.992	0.998 / 0.988 / 0.989	0.570 / 0.745 / 0.409
Ransomware	9885	0.990 / 0.994 / 0.988	0.984 / 0.990 / 0.982	0.621 / 0.753 / 0.374
SQL_injection	50981	0.961 / 0.970 / 0.999	0.828 / 0.713 / 0.999	0.764 / 0.871 / 0.787
Uploading	37243	0.665 / 0.652 / 0.527	0.204 / 0.266 / 0.304	0.581 / 0.512 / 0.295
Vulnerability_scanner	50044	0.992 / 0.993 / 0.859	0.980 / 0.894 / 0.335	0.880 / 0.976 / 0.772
XSS	15172	0.943 / 0.991 / 0.867	0.823 / 0.986 / 0.767	0.680 / 0.650 / 0.483

Table 6.10: Zero-day (leave one attack class out), full pool: mean over the held-out classes: per held-out class

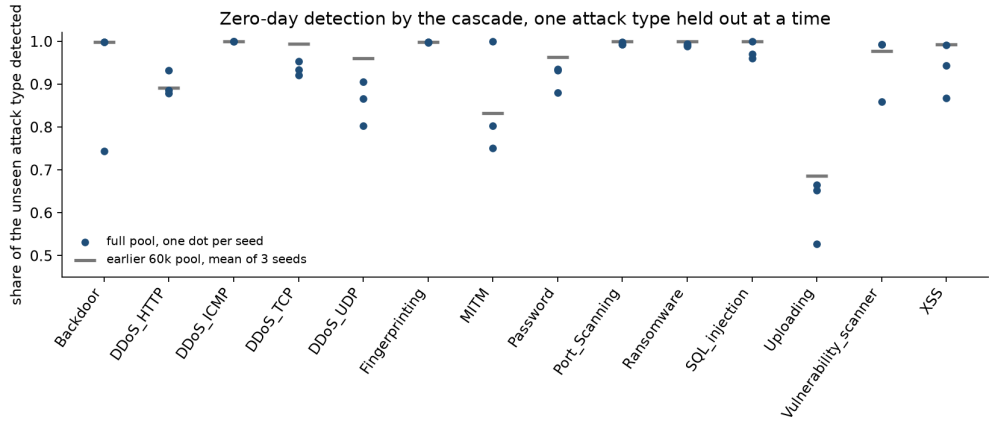


Figure 6.5: Zero-day detection by the cascade at full scale (one dot per seed) and the earlier 60k-pool mean

Stealth attacker, pruning and clipping (seed 42 only). The stealth attacker survives in 0.60, 0.73 and 0.80 of client-rounds against 0.30, 0.50 and 0.60 for sign-flip, with no clear extra harm to the autoencoder and known-attack recall of at least 0.9949 (Table 6.11). Pruning during training costs more at full scale than at 60,000 rows: autoencoder-alone recall falls from 0.457 to 0.350 at 70% sparsity, and to 0.098 when pruned after training (Table 6.12). Without the per-client clip the final benign validation loss is 0.242 against 0.129 with it, much less than the factor of about 1,500 at the smaller scale (Table 6.13).

Attackers	Sign-flip survival	Stealth survival	Sign-flip AE-alone recall	Stealth AE-alone recall	Stealth known-attack recall
20%	0.30 / 0.40 / 0.23	0.60 / n/a / n/a	0.466 / 0.483 / 0.258	0.454 / n/a / n/a	0.9956 / n/a / n/a
30%	0.50 / 0.55 / 0.62	0.73 / n/a / n/a	0.490 / 0.373 / 0.245	0.427 / n/a / n/a	0.9961 / n/a / n/a
40%	0.60 / 0.64 / 0.61	0.80 / n/a / n/a	0.297 / 0.139 / 0.074	0.334 / n/a / n/a	0.9949 / n/a / n/a

Table 6.11: Stealth attacker vs. sign-flip, full pool

Sparsity	Non-zero weights	Autoencoder-alone recall	Cascade macro-recall	Benign FPR
0%	9600	0.457 / 0.492 / 0.221	0.996 / 0.996 / 0.994	0.1182 / 0.0414 / 0.0352
50%	4800	0.439 / n/a / n/a	0.995 / n/a / n/a	0.1400 / n/a / n/a
70%	2880	0.350 / n/a / n/a	0.993 / n/a / n/a	0.1201 / n/a / n/a
80%	1920	0.338 / n/a / n/a	0.993 / n/a / n/a	0.1451 / n/a / n/a
90%	960	0.198 / n/a / n/a	0.993 / n/a / n/a	0.1151 / n/a / n/a

Table 6.12: Pruning during federated training, full pool (no attackers)

Normalisation	seed 42	seed 43	seed 44
clip 10	0.1292	0.05736	0.2644
no clip	0.242	n/a	n/a

Table 6.13: Per-client z-score clip, final-round benign validation loss

Live replay of every capture. All 14 attack captures were replayed through Mininet with the full-pool bundle (Table 6.14). Every one ends in a block decision with the drop rule read back from the switch; 12 of 14 are classified as the right attack type, while the MITM capture is called Ransomware and the OS-fingerprinting capture DDoS_ICMP. The replays are of the dataset's own captures, so they demonstrate the pipeline and are not a held-out test. The released Vulnerability-scanner capture is damaged part-way, so its readable first 265,827 packets were used.

Capture	Classified as	Confidence	Stage	Packets flagged	Action	Wall time
Backdoor attack	Backdoor	0.997	boosting	24,543 / 24,914	block	12 s
DDoS HTTP Flood Attacks	DDoS_HTTP	0.966	boosting	119,748 / 229,142	block	29 s
DDoS ICMP Flood Attacks	DDoS_ICMP	1.000	boosting	2,914,156 / 2,914,356	block	232 s
DDoS TCP SYN Flood Attacks	DDoS_TCP	0.999	boosting	2,020,023 / 2,020,152	block	211 s
DDoS UDP Flood Attacks	DDoS_UDP	0.999	boosting	2,899,758 / 3,215,732	block	268 s
MITM ARP spoofing DNS Attack	Ransomware	0.991	boosting	542 / 1,229	block	8 s
OS Fingerprinting attack	DDoS_ICMP	0.980	boosting	1,134 / 1,176	block	9 s
Password attacks	Password	0.957	boosting	919,469 / 1,053,893	block	130 s
Port Scanning attack	Port_Scanning	0.999	boosting	22,495 / 23,329	block	12 s
Ransomware attack	Ransomware	0.994	boosting	10,620 / 11,030	block	11 s
SQL injection attack	SQL_injection	1.000	boosting	51,097 / 51,228	block	17 s
Uploading attack	Uploading	0.999	boosting	37,592 / 37,644	block	15 s
XSS attacks	XSS	0.999	boosting	15,677 / 16,215	block	12 s
Vulnerability scanner attack	Vulnerability_scanner	0.996	boosting	262,922 / 265,827	block	60 s

Table 6.14: Phase B on every attack capture (10 clients, seed 42, full-pool bundle): class chosen, confidence, deciding stage, packets flagged, action

6.1.3 Training time and time to decide on a packet

Training time. Table 6.15 gives the time of each of the six full-dataset runs (one server and 10 or 20 client processes, 20 rounds), taken from the stage timer saved with each bundle. Federated training takes 82% to 90% of the total. Everything else (loading, the calibration split and bootstrap classifier, threshold calibration, evaluation on 283,090 to 295,566 test rows, saving) takes 175 to 211 s, of which evaluation is 156 to 191 s. For comparison, the stand-alone classifier on all rows trains in 24.2 s (it stopped at tree 54, with 1,429,825 rows for training and 158,870 for early stopping). Some of these runs shared the virtual machine with other jobs, so the times are upper bounds.

Run	Total (s)	Federated training, 20 rounds (s)	All other stages (s)
10 clients, seed 42	1843.6	1632.8	210.8
10 clients, seed 43	1397.2	1220.9	176.3
10 clients, seed 44	1753.6	1578.8	174.8
20 clients, seed 42	1260.3	1072.6	187.7
20 clients, seed 43	1063.4	871.5	191.9
20 clients, seed 44	1225.0	1033.7	191.3

Table 6.15: Time taken by each full-dataset run (`journey.json` of each bundle)

The 20-client runs were faster than the 10-client runs; each client holds about half as many rows, but this was not tested separately, so the cause is not claimed.

Time to decide on one packet. The latency benchmark (`tools/latency_benchmark.py`) loads the saved 10-client, seed-42 bundle and times the models on held-out test packets, on one CPU thread of the virtual machine: 3,000 packets timed one call at a time, then one batch of 200,000. Table 6.16 gives the result.

Mode	Median (ms)	95th (ms)	99th (ms)	Batch (μs per packet)	Batch (packets/s)
Classifier alone	0.372	0.440	0.497	264.2	3,784
Autoencoder alone	0.060	0.094	0.660	5.5	181,382
Cascade (both)	0.642	0.706	0.819	265.9	3,760

Table 6.16: Time for the models to decide on a packet (model time only, one thread)

The classifier dominates the cost. This is the time for the models only: it leaves out reading a capture, TShark feature extraction and the switch. Phase B is not inline, so there is no per-packet blocking latency to report. Averaged over a whole replay (wall time divided by packets, from Table 6.14), the live path took 83 μs a packet for the DDoS UDP capture, 80 μs for DDoS ICMP and 104 μs for DDoS TCP. These numbers come from one bundle, one thread and one machine, and no confidence interval is claimed.

6.1.4 Boosting vs. a deep-learning classifier

On the same 500,000-row sample, LightGBM (base features) reached 0.887 macro F1 and a dense MLP of comparable effort (8,271 parameters, 15 epochs) reached 0.622. The gap was largest on the application-layer attacks (XSS 0.761 vs. 0.072 recall; SQL_injection 0.767 vs. 0.183). This supports the boosting-first design. It is a comparison against a modest baseline, not a claim that deep learning cannot work here.

6.1.5 Earlier results on the 60,000-row pool

The experiments in this subsection were run first, on a 60,000-row federated pool split row by row, with the 91 or 130 features available at the time. They are kept for comparison with the full-dataset results above; where both exist, the full-dataset result is the one to quote. Numbers in square brackets are the values before the service-port, content and flow features were added.

6.1.5.1 Ablation of the federated pipeline

Figure 6.6 and Table 6.17 compare five variants with 30% sign-flip attackers (clients 0, 6 and 9), 20 rounds, with the 130-feature pipeline; the original 91-feature results are given in brackets. Known-attack detection is now close to perfect for every variant that includes the classifier (0.996–1.000 macro-recall), so the backstop's value is on *zero-day* attacks: 0.930 macro-recall for the full pipeline against 0.795 for boosting alone. Without the trust filter (plain FedAvg, trimmed mean only) the autoencoder is poisoned again: its stand-alone recall drops to about 0, its validation loss diverges, and zero-day recall falls back to boosting's level (0.795 and 0.799).

Variant	**Attack MR**	**Benign FPR**	**Detection F1**	**Zero-day MR**	**AE-alone MR**
Full pipeline	1.000 [0.887]	0.031	0.961	0.930 [0.731]	0.712 [0.421]
Plain FedAvg	0.998 [0.879]	0.035	0.955	0.795 [0.595]	0.002 [0.000]
Trimmed mean only	0.997 [0.878]	0.033	0.957	0.799 [0.539]	0.000 [0.000]
Autoencoder only	0.372 [0.191]	0.030	0.465	0.577 [0.325]	0.372 [0.191]
Boosting only	0.996 [0.875]	0.000	0.998	0.795 [0.518]	n/a

Table 6.17: Ablation at 30% malicious clients, 130 features [91 features] (MR = macro-recall)

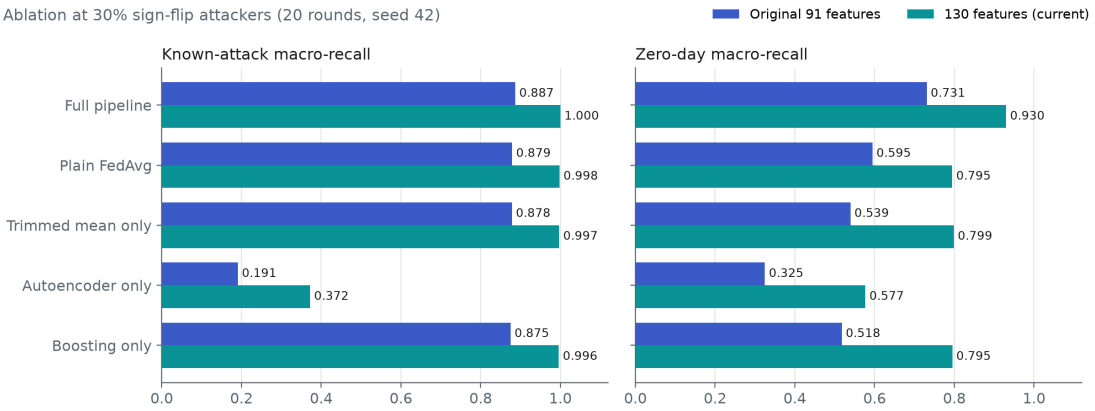


Figure 6.6: Ablation: known-attack and zero-day macro-recall, original vs. current features

One trade-off is worth stating. On known attacks, boosting alone now has a higher detection F1 than the full cascade (0.998 vs. 0.961). The autoencoder's threshold is set to flag about 3% of benign traffic, and that cost now outweighs the little it adds when the classifier already catches 99.6% of known attacks. Final validation losses: full pipeline 0.028; plain FedAvg 5.0×10^{20} and trimmed mean 1.3×10^{4} (diverged). Communication: 15.7 MB over 20 rounds (the autoencoder now has 9,834 parameters, up from 7,299).

6.1.5.2 Poisoning resistance

Figure 6.7 sweeps the share of sign-flip attackers from 0 to 50%. Attack macro-recall stays between 0.997 and 1.000 at every fraction [0.878–0.887], and zero-day macro-recall between 0.901 and 0.943 up to 40% [0.724–0.736]. Attackers got past the filter in 10%, 12% and 36% of attacker-rounds at 20%, 30% and 40% [28%, 32%, 24%]: lower at the first two fractions but higher at 40%, although the autoencoder still held there (stand-alone recall 0.691). At 50% the coordinate-wise median reference becomes the attackers' direction again: every attacker update passes, the autoencoder collapses, and zero-day detection falls back to the classifier (0.803). This is the known limit of median-based defences [10]. A repeat with seeds 43 and 44 at 20% and 30% gave 0.996–0.999 attack macro-recall for every seed, while stand-alone autoencoder recall varied by seed (0.56–0.74), as it did with the original features.

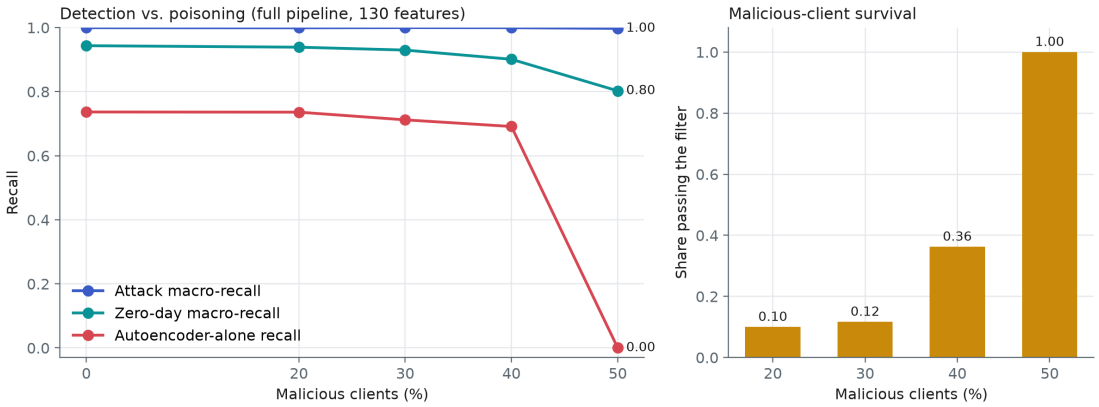


Figure 6.7: Detection and malicious-client survival vs. share of malicious clients (130 features)

6.1.5.3 Zero-day detection

Each of the 14 attack classes was removed in turn from all training data (including scaler statistics) and then scored as unseen traffic (Figures 6.8 and 6.9). The cascade's average detection rose from 0.736 to **0.943**; boosting alone from 0.533 to 0.801 and the autoencoder alone from 0.493 to 0.757. The autoencoder separates unseen attacks from benign traffic far better than before (AUROC 0.968–1.000 for every class, against 0.59–0.70 for the HTTP-family attacks with the original features). The largest gains are on the attacks that were hardest before: Password 0.44→0.97, Ransomware 0.54→1.00, SQL_injection 0.61→1.00, XSS 0.52→0.99 and DDoS_HTTP 0.36→0.88.

Two classes got worse and are reported rather than hidden. MITM fell from 0.89 to 0.69: its single ARP connection gets ordinary-looking connection features, so the autoencoder finds it less unusual. Unseen Uploading traffic now looks normal to the classifier (boosting-only 0.49→0.08); the autoencoder catches most of it, so the cascade still improves (0.52→0.72). The benign false-positive rate stays about 3%, as set by the 97th-percentile threshold.

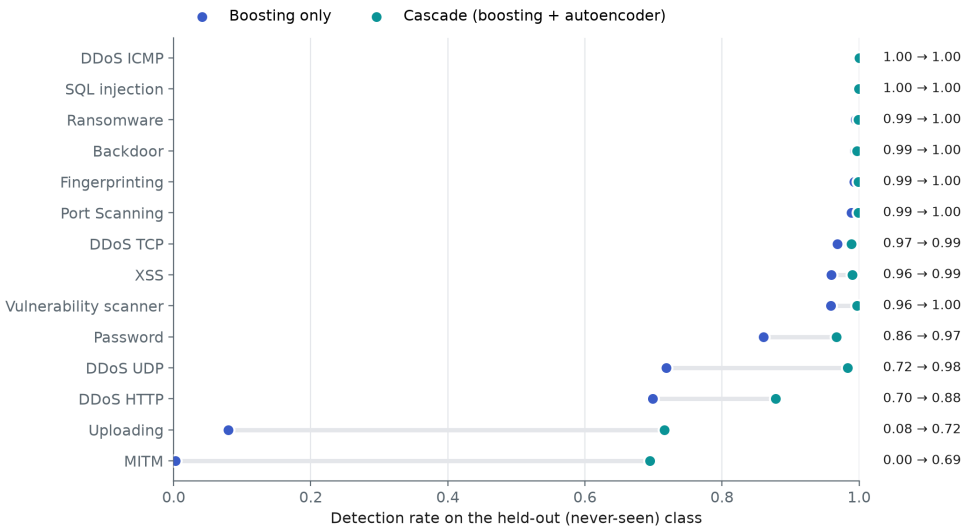


Figure 6.8: Zero-day detection per held-out class: boosting alone vs. cascade (130 features)

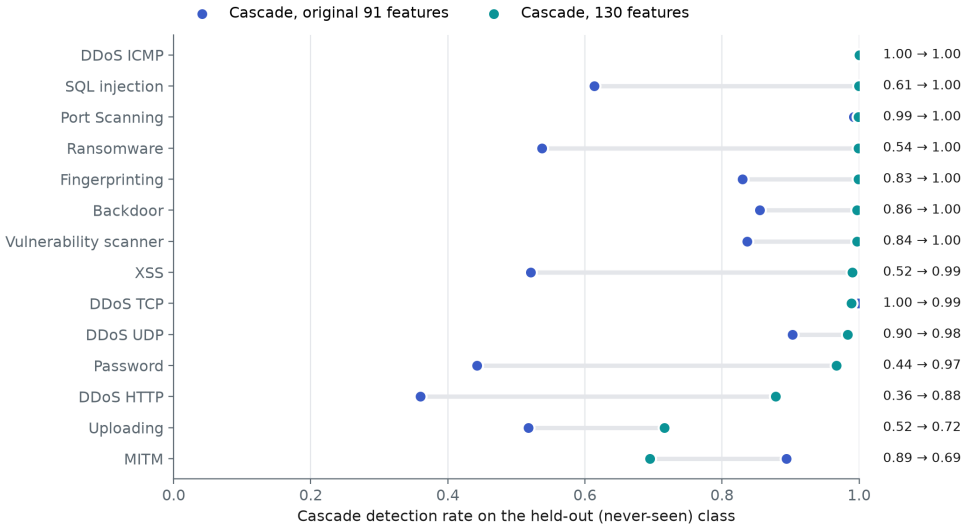


Figure 6.9: Cascade zero-day detection per held-out class: original 91 features vs. 130 features

6.1.5.4 Real multi-process run and SDN mitigation

Phase A, real run. A real Flower server and 10 separate client processes completed 20 rounds in about 4 minutes. The bootstrap classifier reached 0.997 accuracy on the calibration hold-out [0.963]. The final cascade gave **0.999** attack macro-recall [0.889] and a 0.035 benign false-positive rate [0.035] on the clients' test slices. A separate full-pipeline run scored per class gave 0.976 accuracy and 0.971 macro F1 for the cascade [0.944 / 0.931], and an autoencoder AUROC of 0.982 [0.805].

Phase B. Live extraction now derives the same 130 features from the capture with the same code as training: for `Backdoor_attack.pcap`, 99.8% of the live rows are identical to that capture's training rows. Replayed through Mininet with the new bundle, the capture was classified as **Backdoor with 0.997 confidence** by the boosting stage, 24,669 of its 24,914 packets were flagged [23,895], the bridge chose *block*, and the rule `priority=100,ip,nw_src=10.0.0.1 actions=drop` was confirmed in the switch's flow table. Because classification happens once features have been extracted from the whole capture, 23,161 replayed packets had already been forwarded when the rule appeared. The mitigation is therefore capture-level, not inline per-packet blocking.

6.1.5.5 Scaling to 20 clients, repeat seeds and run timing

Real 20-process runs. The project was re-hosted on a virtual machine with 24 GB of memory, which can hold one Flower server and 20 client processes (an earlier attempt on a 7.4 GB machine could not). Phase A was run with 10 and with 20 real client processes at seeds 42, 43 and 44 (20 rounds, no attackers). The seed changes the data split, the Dirichlet partition and the initialisation. Table 6.18 lists every run. Over the three seeds the cascade's attack macro-recall averages 0.998 with 10 clients and 0.992 with 20 clients, and the benign false-positive rate stays between 2.8% and 4.0%. At seed 42, the 20-client run dips to 0.978; this run was not analysed per class, so its cause is not established. With only three seeds, no confidence intervals are claimed.

Run	Phase A (s)	Attack macro-recall	False alarms	Classifier acc.
10 clients, seed 42	341	0.999	3.40%	0.9969
10 clients, seed 43	276	0.999	3.11%	0.9976
10 clients, seed 44	262	0.996	2.84%	0.9968
20 clients, seed 42	247	0.978	3.62%	0.9969
20 clients, seed 43	238	0.999	2.84%	0.9976
20 clients, seed 44	242	0.999	3.96%	0.9968

Table 6.18: Real multi-process Phase A runs (source: `results/real_runs/summary.md`)

Phase B on the 20-client bundle classified `Backdoor_attack.pcap` as Backdoor with 0.997 confidence, flagged 24,466 of 24,914 packets, and the drop rule `priority=100,ip,nw_src=10.0.0.1 actions=drop` was read back from the switch.

The trust filter without attackers. Even with no malicious clients, the filter excluded some clients' updates: over 20 rounds, 15, 2 and 6 client-rounds with 10 clients,

and 38, 42 and 27 with 20 clients (seeds 42, 43, 44). These are honest clients whose non-IID updates look unlike the median, so the filter cannot tell unusual data from sabotage. This is the confound named in the limitations, and it is more frequent with 20 clients.

Simulation versus real run. An in-process simulation of 20 clients (seed 42) gave a cascade attack-detection recall of 0.9960 against 0.9996 for 10 clients, an autoencoder recall of 0.685 against 0.728, and benign false alarms of 3.72% against 3.32%. Three of the 20 simulated clients received no benign validation rows, so their anomaly threshold is infinite and their autoencoder never flags. The simulation uses the same client, strategy and trust-filter code as the real run, and the table above is the real-process evidence.

Timing of one complete run. One real run (10 client processes, warm feature cache) took 219 s in total: Phase A 210.6 s, of which federated training was 191.2 s, and Phase B 8.6 s. Tables 6.19 and 6.20 give the stages.

Phase A stage	**Seconds**
Load, clean and encode the dataset	1.9
Calibration split, bootstrap classifier, client partition	14.4
Federated training	191.2
Calibrate client anomaly thresholds	0.1
Evaluate on the clients' test slices	2.5
Save the model bundle	0.2

Table 6.19: Phase A stage timing (source: `results/journey/phase_a_journey.json`)

Phase B stage	**Seconds**
Load the model bundle	0.08
Start the SDN bridge and controller	2.20
Network, replay, classify and block (root process)	6.33
Check the switch's flow table	0.00

Table 6.20: Phase B stage timing (source: `results/journey/phase_b_journey.json`)

6.1.5.6 Follow-up experiments: more seeds, an adaptive attacker, pruning and a time-ordered split

These experiments were run after the main results, in the virtual machine, and every table is generated from the experiments' own result files.

More seeds. The ablation, the poisoning sweep and the zero-day experiment were repeated at seeds 43 and 44 (Tables 6.21–6.23). The conclusions hold in all three seeds: the full pipeline's attack macro-recall stays between 0.995 and 1.000 up to 50% attackers; plain FedAvg and the trimmed mean alone leave the autoencoder at recall 0.000–0.026 with 30% attackers; the trust filter fails at 50% (attacker survival 0.97–1.00); and the cascade's mean zero-day detection is 0.943, 0.952 and 0.952. Other numbers move with the seed: the autoencoder's recall at 30% attackers ranges from 0.565 to 0.712, attacker survival at 20% from 0.03 to 0.23, and MITM zero-day detection from 0.695 to 0.963. The per-class zero-day columns inside the ablation and sweep tables were not repeated at seeds 43 and 44 (about 85 extra trainings per seed). Three seeds are too few for confidence intervals.

Variant	**Attack macro-recall**	**Autoencoder-alone recall**	**Benign FPR**
full_pipeline	1.000 / 0.999 / 0.997	0.712 / 0.679 / 0.565	0.0309 / 0.0284 / 0.0320
plain_fedavg	0.998 / 0.997 / 0.994	0.002 / 0.000 / 0.000	0.0347 / 0.0317 / 0.0297
trimmed_mean_only	0.997 / 0.997 / 0.996	0.000 / 0.000 / 0.026	0.0334 / 0.0291 / 0.0337
autoencoder_only	0.372 / 0.295 / 0.306	0.372 / 0.295 / 0.306	0.0301 / 0.0281 / 0.0325
boosting_only	0.996 / 0.997 / 0.996	n/a / n/a / n/a	0.0002 / 0.0000 / 0.0005

Table 6.21: Ablation at 30% sign-flip attackers, seeds 42 / 43 / 44 (source: results/seeds/summary.json)

Attackers	**Attack macro-recall**	**Autoencoder-alone recall**	**Attacker survival**
0%	1.000 / 0.999 / 0.997	0.737 / 0.765 / 0.612	n/a / n/a / n/a
20%	0.999 / 0.999 / 0.996	0.736 / 0.739 / 0.618	0.10 / 0.03 / 0.23
30%	1.000 / 0.999 / 0.997	0.712 / 0.679 / 0.565	0.12 / 0.12 / 0.27
40%	1.000 / 0.998 / 0.997	0.691 / 0.620 / 0.537	0.36 / 0.31 / 0.44
50%	0.997 / 0.997 / 0.995	0.000 / 0.000 / 0.000	1.00 / 0.99 / 0.97

Table 6.22: Poisoning sweep, seeds 42 / 43 / 44

Stage	**Mean detection rate**
Classifier alone	0.801 / 0.748 / 0.791
Autoencoder alone	0.757 / 0.775 / 0.747
Cascade	0.943 / 0.952 / 0.952
Cascade, DDoS_HTTP	0.878 / 0.846 / 0.948
Cascade, MITM	0.695 / 0.963 / 0.837
Cascade, Uploading	0.716 / 0.677 / 0.665

Table 6.23: Zero-day detection of the unseen class, mean over 14 classes, seeds 42 / 43 / 44

Why the 20-client run dipped. The seed-42 run with 20 real clients scored attack macro-recall 0.978, against 0.996–0.999 elsewhere. The cause is one missed Fingerprinting test row out of four; micro recall is 0.9964 (Table 6.24). Clients without benign validation rows (infinite threshold) were still caught by the classifier.

Run	**Macro-recall**	**Micro recall**	**Fingerprinting test rows, recall**	**MITM test rows, recall**	**Clients with infinite threshold**
10 clients, seed 42	0.999	0.9992	3 rows, 1.00	3 rows, 1.00	0
10 clients, seed 43	0.999	0.9992	4 rows, 1.00	4 rows, 1.00	1
10 clients, seed 44	0.996	0.9960	4 rows, 1.00	5 rows, 1.00	1
20 clients, seed 42	0.978	0.9964	4 rows, 0.75	1 rows, 1.00	3
20 clients, seed 43	0.999	0.9992	2 rows, 1.00	3 rows, 1.00	2
20 clients, seed 44	0.999	0.9988	2 rows, 1.00	3 rows, 1.00	2

Table 6.24: Where the real-run macro-recall comes from (source: results/real_runs/¡run¿/, made by fl_ids.eval.bundle_report)

Real attackers. With sign-flip attackers in the real multi-process run, the trust filter excluded the attackers' updates in 36 of 40 (20%) and 52 of 60 (30%) client-rounds, and excluded no honest client (Table 6.25, Figure 6.10).

Attackers	**Attacker clients**	**Attacker client-rounds excluded**	**Honest client-rounds excluded**	**Attack macro-recall**	**Benign FPR**
20%	0,7	36 of 40	0 of 160	0.998	0.0335
30%	0,6,9	52 of 60	0 of 140	0.999	0.0317

Table 6.25: Real multi-process Phase A with sign-flip attackers (10 clients, 20 rounds, seed 42; source: runs/attack_runs/)

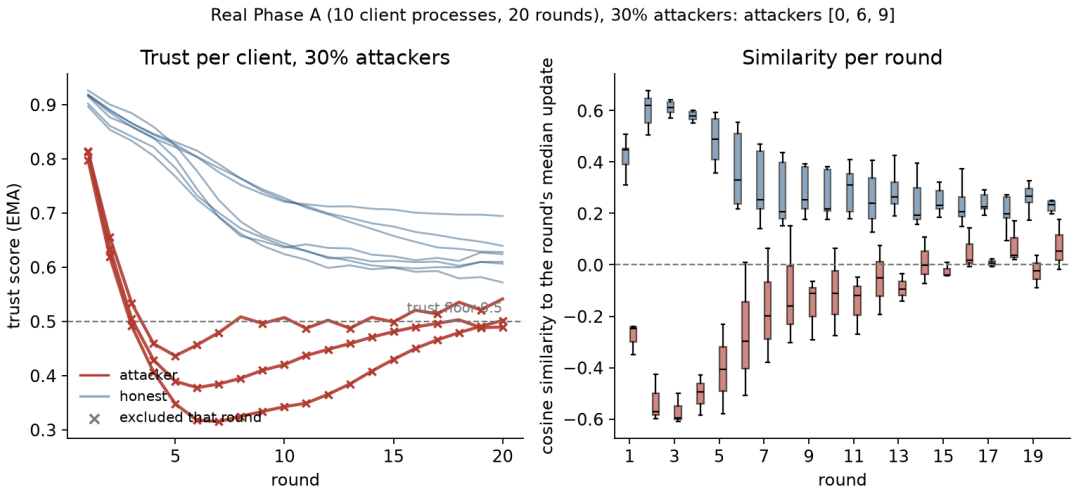


Figure 6.10: Real run with 30% sign-flip attackers: trust score (left) and cosine similarity to the round's median update (right)

An adaptive attacker. A white-box attacker that knows the filter's rule estimates the consensus from the last aggregate update, keeps a cosine similarity of 0.1 to it, and spends the rest of a norm budget (1.5 times its honest norm, under the clip) on the opposite of its honest update's orthogonal component. The filter mostly fails against it: attackers survive in 72–90% of client-rounds instead of 10–36% (Table 6.26). The damage is bounded: known-attack recall stays at 0.9997 or above, but the autoencoder's recall alone falls to 0.499 at 40% attackers (0.663 for sign-flip). Zero-day detection under this attacker was not measured, and this is one adaptive strategy, not a proof for all.

Attack	Attackers	Attack macro-recall	Autoencoder-alone recall	Attacker survival	Final validation loss
sign_flip	20%	0.9993	0.730	0.10	0.026
sign_flip	30%	1.0000	0.718	0.12	0.029
sign_flip	40%	0.9997	0.663	0.36	0.021
stealth	20%	0.9997	0.747	0.90	0.041
stealth	30%	1.0000	0.741	0.72	0.040
stealth	40%	0.9997	0.499	0.82	0.037

Table 6.26: Adaptive (stealth) attacker vs. sign-flip, seed 42, 20 rounds (source: results/adaptive/adaptive_attack.csv)

Weight pruning (Objective 3). The autoencoder's weight matrices were magnitude-pruned after training and during federated training (the server re-prunes every round). Pruning after training damages the backstop at once, while pruning during training keeps it up to about 70% sparsity: autoencoder-alone recall 0.744 dense, 0.727 at 50%, 0.688 at 70%, then 0.467 at 80% (Tables 6.27 and 6.28). A sparse layout needs index storage, so the size falls below the dense size only past 50% sparsity (0.61 of the dense bytes at 70%; computed from the non-zero counts,

not transmitted). Clients still send dense updates, so this reduces the model, not the per-round communication. Single seed, no attackers.

Sparsity	Non-zero weights	Sparse / dense bytes	Autoencoder recall	Autoencoder AUROC	Cascade macro-recall	Benign FPR
0%	9600	1.98	0.742	0.987	0.999	0.0340
50%	4800	1.00	0.273	0.742	0.998	0.0287
70%	2880	0.61	0.168	0.596	0.998	0.0292
80%	1920	0.41	0.145	0.559	0.996	0.0334
90%	960	0.22	0.144	0.594	0.996	0.0323
95%	480	0.12	0.147	0.593	0.996	0.0366
98%	192	0.06	0.147	0.593	0.996	0.0366

Table 6.27: Post-training magnitude pruning of the 10-client autoencoder, thresholds recalibrated (source: results/pruning/c10_s42/pruning.csv)

Sparsity	Non-zero weights	Sparse / dense bytes	Autoencoder-alone recall	Cascade macro-recall	Benign FPR	Final validation loss
0%	9600	1.98	0.744	1.000	0.0320	0.034
50%	4800	1.00	0.727	1.000	0.0323	0.040
70%	2880	0.61	0.688	1.000	0.0298	0.049
80%	1920	0.41	0.467	0.999	0.0312	0.080
90%	960	0.22	0.176	0.998	0.0289	0.207

Table 6.28: Pruning during federated training (server re-prunes each round, linear ramp over the first 10 rounds), seed 42, no attackers (source: results/pruning/pruning_during_fl.csv)

Clipping. Without the per-client z-score clip the benign validation loss stays near 50; with it, it falls from 0.2047 to 0.03369 (Table 6.29, Figure 6.11).

Normalisation	Round 1	Round 5	Round 10	Round 20
clip 10	0.2047	0.04336	0.03999	0.03369
no clip	50.63	50.15	50	49.86

Table 6.29: Benign validation loss with and without the per-client z-score clip (source: results/clip/clip_comparison.csv)

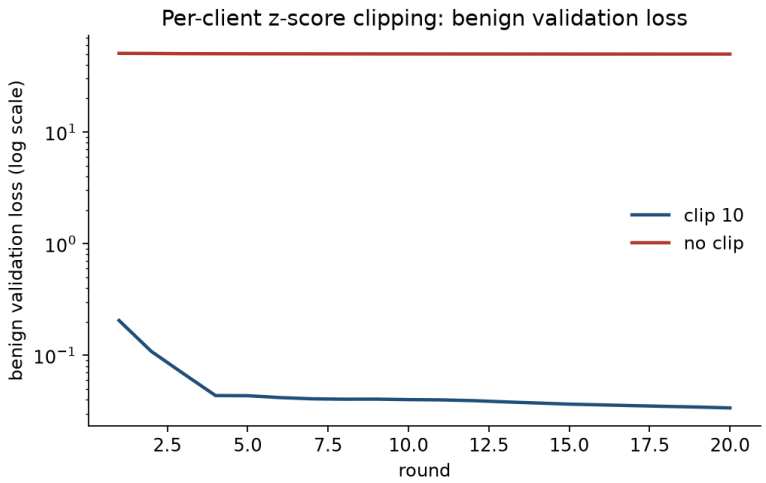


Figure 6.11: Benign validation loss with and without the per-client z-score clip

A time-ordered split. Each class's earliest 80% of connections were used for training and its latest 20% for testing (the CSV is in capture order within a class, but its timestamps carry no date, so this orders by row position). Accuracy is 0.994 against 0.997 for the connection-grouped split (Table 6.30). Macro F1 falls from 0.983 to 0.913, mostly because MITM is a single connection, so it falls entirely in the test side and enters the mean with recall 0; without it the figure is about 0.978. The real losses are Backdoor (0.989 to 0.947–0.962) and DDoS_HTTP (0.980 to 0.912–0.925).

Split	**Accuracy**	**Macro F1**
Time-ordered, seeds 42 / 43 / 44	0.9942 / 0.9943 / 0.9941	0.9133 / 0.9122 / 0.9129
Connection-grouped, seed 42	0.9972	0.9828

Table 6.30: Per-class time-ordered split vs. connection-grouped split (source: experiments/accuracy/results/full_temporal_bal_s*.json)

The placeholder leak, measured. Before the fix, the four affected raw columns separate attack from normal with AUROC 0.9998–1.0. On the current 130-feature matrix the strongest single feature is the flow's reset count (0.879), followed by its packet count (0.866) and the service port (0.835); these are connection summaries rather than spelling artifacts (Table 6.31, Figure 6.12).

Feature	**AUROC**	**State**
mqtt.topic == 0.0	1.0000	raw, before the fix
mqtt.conack.flags == 0.0	1.0000	raw, before the fix
mqtt.protoname == 0.0	1.0000	raw, before the fix
dns.qry.name.len == 0.0	0.9998	raw, before the fix
f_rst	0.8789	130-feature matrix, after
f_npkts	0.8657	130-feature matrix, after
svc_port	0.8350	130-feature matrix, after
f_fin	0.8039	130-feature matrix, after
f_pl_nonzero	0.7328	130-feature matrix, after
f_syn	0.7108	130-feature matrix, after

Table 6.31: Single-feature AUROC, attack vs. normal (source: results/leak/single_feature_auroc.json; after: 300,000-row sample)

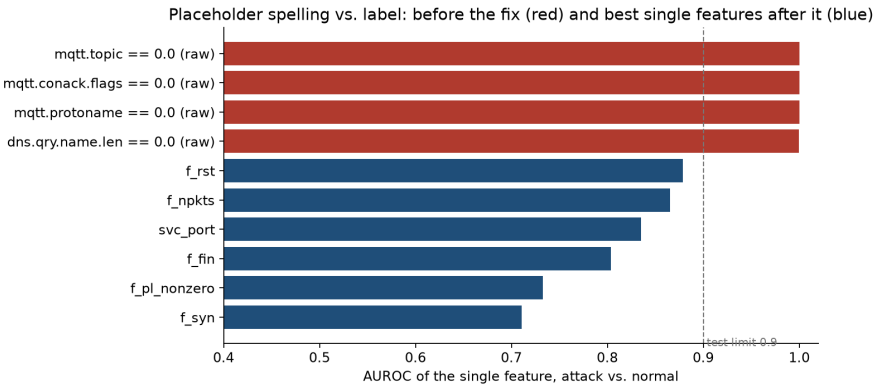


Figure 6.12: Single-feature AUROC before the placeholder fix (red) and best single features after it (blue)

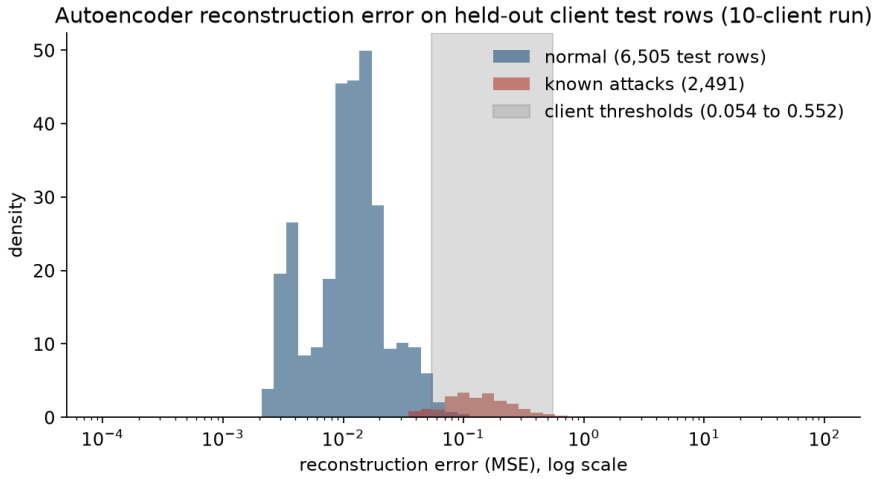


Figure 6.13: Autoencoder reconstruction error on held-out client test rows (10-client run); grey band: range of the clients' thresholds

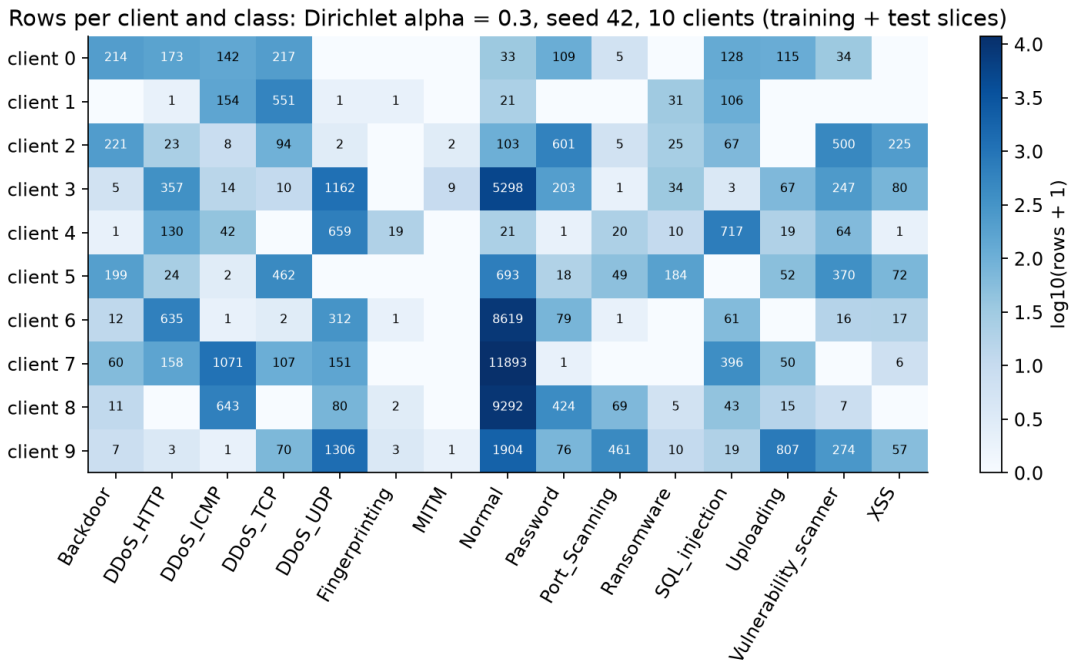


Figure 6.14: Rows per client and class for the 10-client run (Dirichlet $\alpha = 0.3$, seed 42)

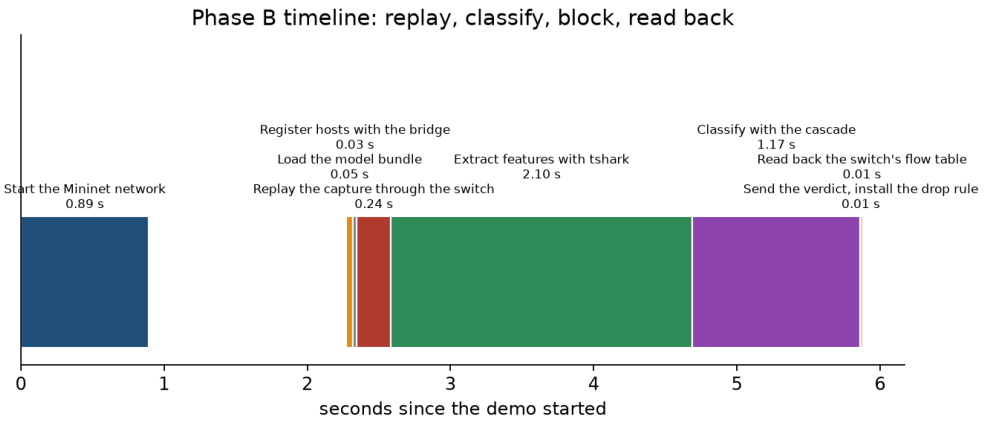


Figure 6.15: Phase B timeline for one replay of the Backdoor capture

6.2 Limitations of the Study

1. **One dataset, one testbed.** Every number comes from Edge-IIoTset. Its benign traffic is only MQTT and VNC, so the service-port feature is partly a testbed effect: in a network with benign web traffic the port alone would separate far less.

2. **Mitigation is not inline.** Phase B classifies a capture after TShark has extracted features from all of it; for a replayed file nearly the whole capture had already passed through the default forwarding entry before the drop rule appeared. No per-packet blocking latency is measured or claimed, and the latency benchmark covers model time only (Section 6.1.3).

3. **Flow features describe whole flow records** (up to 120 s), so the classifier labels flow records rather than deciding on a first packet.

4. **The trust filter is weaker at full scale.** Attacker survival at 30% malicious clients is 0.50 to 0.62 on the whole dataset (0.12 to 0.27 on the 60,000-row pool); the cause is not established. The filter breaks down at 50%, as median-based references do, and a white-box stealth attacker is mostly missed (survival 0.60 to 0.80 at full scale, 0.72 to 0.90 earlier). Only two attack models were tested.

5. **The autoencoder backstop is weaker at full scale**: autoencoder-alone recall is 0.22 to 0.49 on the whole dataset. Clients that hold no benign validation rows (up to three of 20) get an infinite threshold, so their autoencoder never flags anything.

6. **False alarms.** The 97th-percentile threshold gives about 3% benign false alarms by construction. On the whole dataset 2.5% to 16.3% of normal packets are

flagged, but only 0.34% to 1.04% of normal connections, because a few long normal connections dominate the packet count.

7. **Small classes.** MITM is a single connection (406 rows) and cannot be split, so it is scored only by the zero-day experiment and the live replay; Fingerprinting has 872 rows. Per-class figures for these rest on very few test rows.

8. **Zero-day detection depends on the class.** Uploading is the weakest held-out class (0.527 to 0.665), then MITM (0.751 to 1.000) and, depending on the seed, Backdoor and Vulnerability_scanner.

9. **Live replay.** The captures are the dataset's own, the released Vulnerability-scanner capture is damaged (its readable prefix was used), and two of the 14 captures are classified as the wrong attack type (both are still blocked).

10. **Seeds.** The main experiments use three seeds, but the stealth-attacker, pruning and clipping tests at full scale use one, and no confidence intervals are given. Seed-to-seed variation is real (for example autoencoder-alone recall of 0.221 to 0.492 at 0 attackers).

11. **Privacy relaxation.** The incremental classifier update sends at most 50 analyst-confirmed alert rows per surviving client to the server. The calibration set is also held centrally, so the classifier is not federated.

12. **Pruning** reduces the model but clients still send dense updates, so no communication saving is demonstrated.

13. **Emulation.** Clients are processes on one machine and the network is Mininet inside one virtual machine; there are no physical devices, hardware switch or network delays. Timings come from one machine and are upper bounds where other jobs shared it.

14. **Comparability.** The dataset corrections (placeholder fix, MITM repair, connection split) mean the numbers cannot be compared directly with papers that train on the released CSV with a random split; the claim that those results are inflated is a reasoned expectation, not something this report measured on their pipelines.

15. **Novelty** rests on targeted searches only, not a systematic literature review.

16. **Deep-learning comparison.** The comparison of LightGBM with a neural classifier used a 500,000-row class-capped sample and a modest network, so only its relative result is meaningful.

6.3 Conclusion

The results answer the four objectives as follows.

Objective	What the measurements show
1. Malicious-client detection and filtering	Known-attack recall stays at 0.985 to 0.997 up to 50% sign-flip clients on the whole dataset. At 30% the filtered autoencoder keeps 0.245 to 0.490 recall alone, where plain FedAvg falls to 0.002 to 0.092. The filter lets more attackers through than at 60,000 rows and misses a stealth attacker.
2. Efficient, privacy-aware FL	Clients send only weights (9,834 parameters, about 39 kB) and error histograms; a 10-client run on 1.89 million rows takes 23 to 31 minutes and a 20-client run 18 to 21 minutes. Raw traffic stays on the client apart from the capped confirmed alerts.
3. Model optimisation (pruning)	Pruning during training costs autoencoder-alone recall (0.457 dense, 0.350 at 70% sparsity, 0.198 at 90%) while cascade macro-recall stays at 0.993 or above; no communication saving is shown because updates are still dense.
4. Semi-supervised, anomaly-based detection	On attack classes held out of all training, the cascade detects 0.895 to 0.944 on average, against 0.723 to 0.757 for the classifier alone and 0.590 to 0.728 for the autoencoder alone.

Table 6.32: Objectives and the evidence for each (whole dataset, three seeds unless stated)

Beyond the objectives, the stand-alone classifier reaches 99.72% accuracy and 0.983 macro F1 on 397,073 test packets from connections it never saw, all 14 attack captures are blocked by a drop rule read back from the switch, and the models decide on a packet in a median 0.642 ms.

Chapter 7 Conclusion

Conclusion

The five parts of FL-IDS work together on real IoT/IIoT traffic: a boosting classifier, a federated autoencoder backstop, a cosine-similarity trust filter, Flower-based federated learning, and SDN mitigation. Each part has a measured contribution, and the measurements were repeated on the whole Edge-IIoTset dataset, split by connection, rather than on a subsample split by row. All planned phases are complete.

- **Accuracy target met.** With service-port, payload-content and flow features and square-root class weights, the LightGBM classifier reaches **99.7% accuracy and 0.983–0.985 macro F1** when trained on 1.43 million packets and tested on 397 thousand packets from connections that never appear in training. That is up from 96.4% and 0.889 with the original features. It holds across three seeds, and when the classifier is trained on only the 5% server calibration set (99.7% / 0.979).
- **The whole federated system works at full scale.** In six real multi-process runs (10 and 20 clients, three seeds, 1.89 million training rows) the cascade reaches 0.994–0.998 attack macro-recall. The trust-filtered autoencoder stays usable with up to 50% sign-flip clients for known attacks, while plain FedAvg's autoencoder collapses.
- **The backstop pays off on unseen attacks.** When each attack class is held out of training in turn, the cascade detects 0.895–0.944 of the unseen class on average, against 0.723–0.757 for the classifier alone.
- **Detection drives enforcement.** All 14 attack captures replayed through Mininet end in a block decision and a drop rule read back from the switch; 12 of 14 are classified as the right attack type.
- **The models are fast; the pipeline is not inline.** A packet is decided in a median 0.642 ms (one thread), and a whole full-dataset training run takes 18 to 31 minutes. Mitigation is nonetheless capture-level, not per-packet.

- **Data quality.** The released Edge-IIoTset CSV contains a placeholder label leak (one feature with an AUROC of 1.0), first reported by Galal [13], and column-shifted MITM rows, which we found and repaired. Results trained directly on it with a random split are likely inflated.

The limitations that qualify these results are listed in Section 6.2.

Future Work

1. Make mitigation streaming: classify flows incrementally (for example after the first n packets) so a rule can be installed while an attack is still running, and measure the end-to-end blocking latency.

2. Defend against the white-box stealth attacker (Section 6.1.5.6), which the filter mostly misses, for example with a small trusted root dataset as in FLTrust [5]; measure zero-day detection under it; and test other adaptive strategies and attackers above 50%.

3. Find out why the trust filter excludes fewer attackers and the autoencoder is weaker at full scale, and repeat the stealth-attacker, pruning and clipping experiments at full scale over more seeds, with confidence intervals.

4. Validate on a second dataset that contains benign web traffic, to measure how much the service-port feature depends on the testbed.

5. Reduce the per-round communication as well: clients still send dense updates, so sparse updates and zero-day detection of pruned models remain to be tested.

6. Federate the classifier itself (for example with histogram-based federated boosting) so the calibration set no longer needs to be held centrally.

7. Deploy on physical edge hardware and a hardware OpenFlow switch, and measure real network delays.

REFERENCES

[1] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras and H. Janicke, "Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning", *IEEE Access*, vol. 10, pp. 40281–40306, 2022, doi: 10.1109/ACCESS.2022.3165809.

[2] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh *et al.*, "Federated-Learning-Based Anomaly Detection for IoT Security Attacks", *IEEE Internet of Things Journal*, vol. 9, no. 4, 2022.

[3] B. Li, Y. Wu, J. Song, R. Lu, T. Li and L. Zhao, "DeepFed: Federated Deep Learning for Intrusion Detection in Industrial Cyber–Physical Systems", *IEEE Transactions on Industrial Informatics*, vol. 17, no. 8, Aug. 2021.

[4] J. L. Hernandez-Ramos, G. Karopoulos, E. Chatzoglou, V. Kouliaridis, E. Marmol, A. Gonzalez-Vidal and G. Kambourakis, "Intrusion Detection Based on Federated Learning: A Systematic Review", *ACM Computing Surveys*, vol. 57, no. 12, 2025, doi: 10.1145/3731596.

[5] X. Cao, M. Fang, J. Liu and N. Z. Gong, "FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping", in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2021.

[6] J. Xu, S.-L. Huang, L. Song and T. Lan, "Byzantine-robust Federated Learning through Collaborative Malicious Gradient Filtering" (SignGuard), in *Proc. IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2022.

[7] H. Zhang, L. Ge, G. Zhang, J. Fan, D. Li and C. Xu, "A two-stage intrusion detection method based on light gradient boosting machine and autoencoder", *Mathematical Biosciences and Engineering*, vol. 20, no. 4, 2023, doi: 10.3934/mbe.2023301.

[8] Z. Abou El Houda, A. S. Hafid and L. Khoukhi, "MiTFed: A Privacy Preserving Collaborative Network Attack Mitigation Framework Based on Federated Learning Using SDN and Blockchain", *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 4, pp. 1985–2001, 2023.

[9] B. McMahan, E. Moore, D. Ramage, S. Hampson and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data", in *Proc. International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

[10] D. Yin, Y. Chen, K. Ramchandran and P. Bartlett, "Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates", in *Proc. International Conference on Machine Learning (ICML)*, 2018.

[11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[12] T.-M. H. Hsu, H. Qi and M. Brown, "Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification", arXiv:1909.06335, 2019.

[13] M. M. Galal, "Provenance, Not Behaviour: A Serialisation Artifact in Edge-IIoTset and a Leakage-Free Benchmark for Precision-Agriculture Intrusion Detection", arXiv:2608.15761 (preprint), Aug. 2026.

[14] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization", in *Proc. International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.

Plagiarism

The similarity report and the AI-generated-content report are to be attached here after the similarity check.

I Appendix A: Snapshots

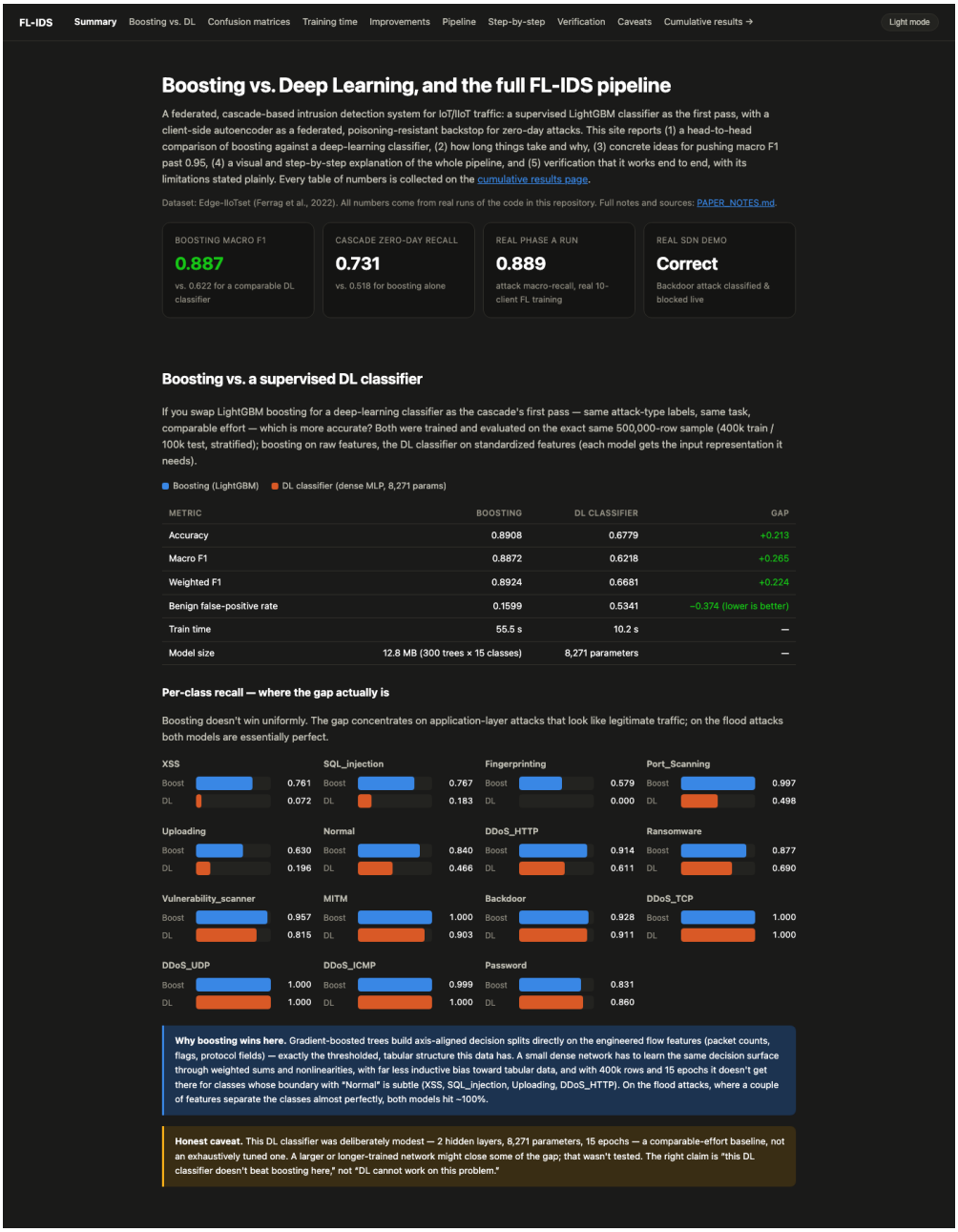


Figure 1: Project results website (`https://aronnnn1.github.io/FL-IDS/`). A second, presentation-style site with animated diagrams is at `https://aronnnn1.github.io/FL-IDS-showcase/`

Phase B result (`runs/phase_b/phase_b_result.json`, excerpt):

```
"classification": "Backdoor", "confidence": 0.9974,
"stage": "boosting", "num_packets": 24914,
"num_non_benign": 23895
```

```
"mitigation": {"action": "block", "applied": true,
               "ip_address": "10.0.0.1"}
flow table:
 priority=100,ip,nw_src=10.0.0.1 actions=drop
 priority=0 actions=NORMAL   (n_packets=23586)
"check": {"action": "block", "consistent": true}
```

Automated test suite:

```
244 passed, 1 skipped, 2 warnings in 462.87s (0:07:42)
```

II Appendix B: Details of list of publications related to this project

Author Names: Aaron B Ajay, Aaryan Prakash, Amal Roy, Amogh Krishna JS, Syed Akram

Paper Title: Federated Learning-based Intrusion Detection System for SDN-enabled IoT Networks (working title)

Name of the Conference or Journal: Paper in preparation (draft in progress)

Place of the conference: –

Date of Conference: –

III Appendix C: Details of Patent

No patent has been filed for this work.

IV Appendix D: Details of Funding

This project did not receive external funding.

V Appendix E:POs and PSOs Mapped

PROGRAMME OUTCOMES	Level (1/2/3)	Justification if addressed
PO1 (Engineering knowledge)	3	Applies mathematics (cosine similarity, robust statistics, gradient boosting) and networking and machine-learning fundamentals to detecting attacks in IoT/IIoT traffic.
PO2 (Problem analysis)	3	Reviews the literature to identify gaps, and analyses why accuracy was stuck (content-free packets, ambiguity ceiling) and why aggregation fails under poisoning, with measured evidence.
PO3 (Design/development of solutions)	3	Designs a two-stage cascade, a trust-filtered federated aggregation scheme and an SDN mitigation path; keeps raw traffic on each site for privacy, which protects the safety of critical infrastructure.
PO4 (Conduct investigations)	3	Designs controlled experiments (ablation, poisoning sweep, zero-day tests, flow-grouped and multi-seed evaluation) and interprets the data to reach valid conclusions.
PO5 (Modern tool usage)	3	Uses LightGBM, PyTorch, Flower, Mininet, Open vSwitch, os-ken, TShark and Streamlit, and states each tool's limitations (for example, capture-level rather than inline mitigation).
PO6 (The engineer and society)	2	Addresses the security of IoT/IIoT systems society depends on, and the privacy concerns of sharing raw network traffic.
PO7 (Environment and sustainability)	1	Supports resilient, sustainable infrastructure (SDG 9); federated training avoids moving bulk traffic to a central site, and the system runs on existing hardware with open-source software.
PO8 (Ethics)	3	Keeps raw traffic on clients, uses a public dataset, reports dataset errors and limitations openly, and reports only numbers produced by logged runs.
PO9 (Individual and team work)	3	Developed by a four-member team, with each member owning components.
PO10 (Communication)	2	Documents the work in this report, a public repository with detailed notes, a results website, a presentation and a paper draft.
PO11 (Project management and finance)	2	Planned and delivered in phases with milestones tracked in version control, at zero cost using free and open-source software and existing hardware.
PO12 (Life-long learning)	2	Required independent learning of federated learning, adversarial robustness and SDN, which are beyond the regular curriculum.

Table 1: Programme Outcomes Mapping with Project

PROGRAMME SPECIFIC OUTCOMES	**Level (1/2/3)**	**Justification if addressed**
PSO1 (Software engineering)	3	Applies software engineering practice: modular design by component, a single configuration file, 244 automated unit and integration tests, version control and reproducible, seeded runs.
PSO2 (Network, mobile and web-based systems)	3	Designs and builds a networked system under realistic constraints: a federated client–server system, an SDN-controlled emulated network, and a web dashboard, all running within the memory limits of a 4-vCPU virtual machine.
PSO3 (Efficient algorithms and code)	3	Designs efficient algorithms (trust filter on weight deltas, trimmed-mean aggregation, flow grouping of 2.2 million packets in about a minute) and effective code, such as one shared cascade implementation used by evaluation, dashboard and SDN.

Table 2: Programme Specific Outcomes Mapping with Project